

ELEC 1601 Foundation of Computer Systems

Notes Instructions

Format(To help you understand my notes better)

*: Not mentioned in this or before class, not necessary information.

Bold: specify a concept

Italic: emphasise

Text in Red: Important notes

Text in Yellow: Examples

Text in Purple: Java Codes and Assembly Language

Author's reminder about this course

- Outline:
 - The project is based on controlling a robot (boe-bot) with an Arduino board and sending information remotely using a communication device.
 - Lecture: Week 2-5 Computer basic knowledge, encoding information, computer memory, boolean algebra, Sequential circuit.
 - Lecture: Week 7-12 AVR Architecture, Assembly Language
 - Tutorial: Materials and questions about lecture topic
 - Lab: Week 2-6
 1. Work on several pre-defined activities programming the Arduino board with certain sensors and actuators.
 2. Design a novel application using those sensors and actuators and create a prototype.
 - Lab: Week 7-13 work on final project, report, demo, presentation.
 - Study strategies by author:
 - New to computer industry? Probably this course will make you more confused. One of the biggest challenges is that you have to read nearly 10000 words materials every week! This course is about reading, theory, and you have to spend a great amount of time in labs. But do not PANIC! If you panic, nothing will succeed.
 - My notes are extremely useful at this point, although there are so many texts in some parts, but I really tried hard to summarise. Take my notes to lectures and write your thoughts on it would be a good strategy.
 - Practice, Practice and Practice. In this course you will face a lot of problems every week. Make sure you understand them. Do not put off anything. Or you will just find that information just begin to pile up and you can't handle it.
 - **If you purchased my notes:**
 - Thank you very much. You have just made a wise decision. I do put a lot of effort in writing them, and they are simple, orderliness, understandable.
- Computer Industry is changing rapidly. I'll try to keep my notes up to date.

Once you fall behind, you'll find it even harder to catch up.

ELEC 1601 Course Contents(2016 S2)

Week	Topic	Navigation
Week1	Introduction & Study Technique	1. Course Senario 2. Objectives and Milestones 3. Learning Strategies 4. Study Techniques 5. Optimal Working Session 6. Level of Computer Systems
Week2	Encoding Information	1. Binary Logic 2. Representing numbers in different bases(2, 8, 10, 16) 3. Encoding Integers: Sign & Magnitude, 2's complement 4. Floating Point 5. Encode set of symbols
Week3	Computer Memory	1. Computer memory 2. Memory Operations 3. Connection between memory and processor 4. Data Storage 5. Arrays(tables) in memory 6. Indirection
Week4	Boolean Logic and Combinational Circuits	1. Boolean Algebra 2. Logic Gates 3. Design Criteria for Digital Circuits 4. Example of combinational circuits
Week5	Sequential Digital Circuits	1. The clock 2. Flip-Flop 3. FSM
Week6	Mid-Term Examination	
Week7	AVR Architecture and Execution Environment	1. The Execution Environment 2. The Execution Cycle 3. The Stack
Week8	AVR Instruction Set Architecture	1. Types of Instruction Sets, CISC/RISC 2. Instruction format of AVR Architecture 3. The assembly language
Week9	Assembly Programs in AVR	1. Assembly Program Basic 2. Create an Execution Program from Assembly code 3. Data Definition 4. Stack and Register Management
Week10	Addressing Modes of AVR Architecture	1. Notations to describe Addressing Modes 2. Addressing Modes in the AVR Architecture
Week11	Translation of High Level Programming Constructs	1. If-else 2. Switch 3. While 4. For
Week12	Subroutine Execution	1. CALL and RET Instructions 2. Parameters and Result 3. Activation Block

Please do not share this note to others without author's permission. © Copyright Reserved.

ELEC 1601 Week 1

Introduction & Study Technique

Navigation

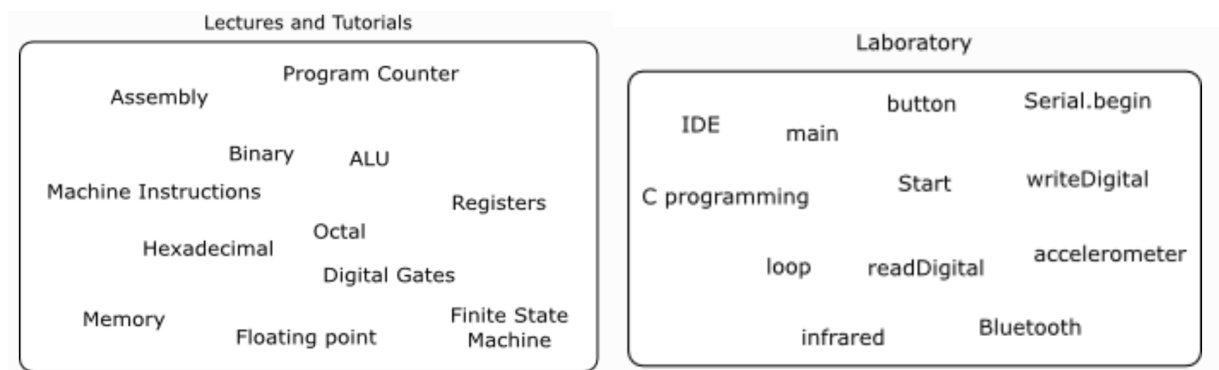
- Course Senario
- Objectives and Milestones
- Learning Strategies
- Study Techniques
- Optimal Working Session
- Level of Computer Systems

Course Senario

You are part of a company department considering the use of digital systems such as Arduino as their next strategic step. But before moving into that space they need to know in detail the technology, its potential, how feasible is to program, and possible applications.

The task for your team is to explore these possibilities. You need to understand how digital systems work, the architecture of Arduino, its machine language, how to program it, how to connect other devices and sensors, how to read and write data from those sensors, become proficient with the tools to carry these tasks, and tests all of them in a practical project.

At the end of the semester you have to explain to the company the possibilities of these systems and propose future applications.



In the lectures and tutorials we will study the system from the point of view of a circuit and the architecture (AVR Architecture) of the microprocessor. How the circuit works and its components. In the lab, we will approach the Arduino from the C-like programming tasks, dealing with the specific syntax to write C programs for Arduino.

Objectives and Milestones

The main objective of the course is to demonstrate how to program and communicate with a digital system.

- Demonstrate that you understand how computers work, from the digital logic level to how they execute basic programs.
- Design, build, configure, program and test an electronic system for a specific engineering problem observing common professional practice.
- Write reports about the design process and its results
- Engage in team-based design and creative tasks to solve an engineering problem.

Milestones

1. How computers encode all information they manipulate in binary logic.

2. How digital circuits are created to manipulate information encoded in binary format.
3. Which instructions are executed by computers and the steps required for that execution.
4. How to write programs in machine language (assembly).
5. How to design and configure a computer system (Arduino) using a high level programming language.

Learning strategies

- System 1(intuition and instantaneous) and System 2 (require rational thinking)
 - learning requires active rational activity, not in System 1 but in System 2.
- learning occurs when there is some degree of difficulty that is overcome
 - **Maximum Learning with Reasonable Effort (MLRE) strategies** are much more effective for academic achievement.
- DEVOTE TIME!
- NO COGNITIVE DEBTS, NO POSTPONE!
- Take the opportunity in this course to refine your electronic device strategy (EDS).

Study Techniques

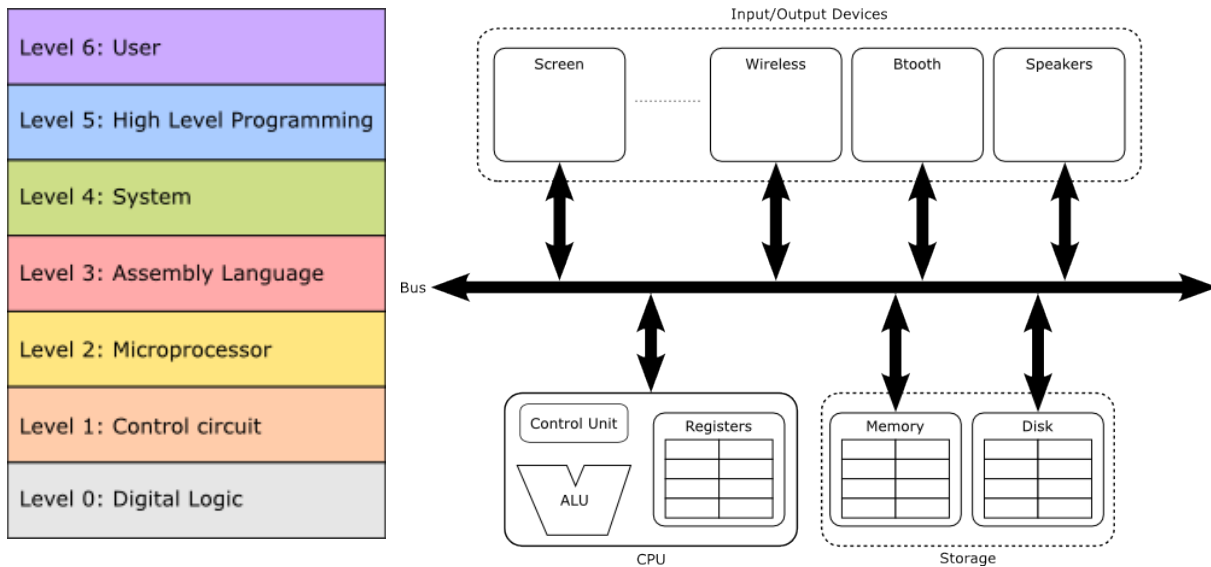
- Avoid System 1
- Avoid person bias: make sure you have learned the content, procedure or concept in a reliable way, not in system 1. Put some time to review; Put some difficulties in the path;
- Avoid massed practice!
- Avoid improvised study sessions
- Distributed Practice Practice Practice
- Interleave topics in a session.
- Keep Asking! Myself and others
- Self-explanation
- Highlighting and Underlining, not FREQUENTLY
- Summarising
- Keyword mnemonic, from complex to easy
- Imagery for text
- Time management
 - Try to pre-allocate working slots. NO INTERRUPTIONS, NO EXCUSES
 - Follow these slots with relaxing slots
 - Check how many of these slots are you able to include in a regular working day and see if you can increase them. What is your maximum? How successful are you keeping them interruption-free? How many did you fail to protect? Would you be able to quantify your cognitive debt in working slots?

Optimal Working Session

1. **Allocate a period of time** of solid work in advance. Make sure nothing interfering.
2. **Sketch a plan** of activities you will try to tackle during that period.
3. **Specify the order of the tasks** and you have all the material. DO NOT SEEK MATERIAL AT THAT TIME.
4. **Start on time.** Disable as many distractions as possible.
5. **Focus on.** If not, draft a precise description of why you cannot finish the task, and the steps you will take once the session is done, to **unblock** this situation.
6. Keep working until time is up. See your notes and try to **assess how much of your cognitive debt has disappeared**, and how much extra time you need to reduce it further.
7. Repeat these steps until the level of cognitive debt is adequate.

Level of Computer Systems

The abstraction levels of computer systems(left), The model of a computer system(right).



ELEC 1601 Week 2

Encoding Information

Navigation

- Binary Logic
- Representing numbers in different bases(2, 8, 10, 16)
- Encoding integers: Sign & Magnitude, 2's complement
- Floating Point
- Encode set of symbols

Binary Logic

A **digital circuit** is a system capable of receiving data encoded with a set of **bits**(inputs), performing some operation, and producing another set of signals(outputs) with the result.

A **microprocessor** is a complex **digital circuit** capable of executing a previously defined set of instructions encoded with bits and known as Machine Language.

But in order for a microprocessor to execute these instructions, all their elements (numbers, symbols, operands, etc.) need to be encoded with **binary logic**.

Encoding a set of elements using binary logic:

1. Using n bits we could encode 2^n elements.
2. Decide how many bits (n) to use if given the number of elements (N)

$$N < 2^n \iff n \geq \log_2 N$$

An encoding may have a number of combinations larger than the number of elements in the set, thus leaving some of the unused.

Once the number of bits has been decided, you need to define a relationship between each element in the set. Each element must have at least one binary representation, and each sequence of bits must correspond with one element in the set.

Representing numbers in different bases(2, 8, 10, 16)

Some basic rules for representing numbers in base b (b can be 2, 3, 9 16... any natural number)

1. The numbers must be less than b . **e.g.** If the number is in base 8, we will not see 8 in the representation, the maximum number is 7.
2. The maximum number with d digits is followed by 1 with $d+1$ digits in which most significant is 1 followed by all zeros. **e.g.** In base 8 we have 777, + 1 we will get 1000.
3. The general formula to obtain the value of any number on any base can be written as:

$$\sum_{i=0}^{n-1} (d_i * b^i)$$

$$\begin{aligned} 6342_7 &= \sum_{i=0}^3 (d_i * 7^i) \\ &= (2 * 7^0) + (4 * 7^1) + (3 * 7^2) + (6 * 7^3) \\ &= 2 + 28 + 147 + 2058 \\ &= 2235_{10}. \end{aligned}$$

4. In order to distinguish the same number from different bases, the base is included in the right side of the number as a subscript. **e.g.** 6342₇ in the example above.
5. The higher the base, the more compact the number representation(i.e. less digits are required).
6. The largest natural number that can be represented with n digits in base b is $b^n - 1$.

From base 10 to base b : Collect the reminders of successive divisions by the new base until the quotient is less than the base. The digits are obtained from least to most significant.

$$\sum_{i=0}^{n-1} (d_i * base^i) = d_0 + (base * \sum_{i=1}^{n-1} (d_i * base^{i-1})) \quad 8675 = (3 * 7^4) + (4 * 7^3) + (2 * 7^2) + (0 * 7^1) + (2 * 7^0)$$

From binary to base 10:

$$\sum_{i=0}^{n-1} (d_i * 2^i) \quad 1101011_2 = 2^6 + 2^5 + 2^3 + 2^1 + 2^0 = 64 + 32 + 8 + 2 + 1 = 107_{10}$$

Some important facts:

- In base 2, if the least significant bit is 1, the number is odd. Otherwise, it is even.
- In any base, Integer multiplication or division by the base is done by adding a zero as least significant digit or removing the least significant bit respectively. **e.g.** For the numbers in base 10, multiply by 10 is done adding a zero as the least significant bit. Analogously, divide by 10, is done by ignoring the least significant digit, which itself corresponds with the remainder of the division.

The base 8(called **octal**) or 16(called **hexadecimal**), can be easily convert from binary without first converting to decimal, because base 8, 16 is a power of 2. ($2^3, 2^4$)

1. Group the number in binary into 3(4) numbers corresponding to the power of 2. Add 0 to most significant bit if there is not enough numbers in a group.
2. Convert each group numbers into decimal.
3. Add all the decimal numbers together.
4. Format. Note that we represent hexadecimal in 0xAA format.

From octal(or hexadecimal) to binary, simply reverse this process. Separating each number into groups of 3(or 4) bits, and then add them together. See table below.

Number in binary	Groups of 3 bits	Number in octal	Groups of 4 bits	Number in hexadecimal
00100111 ₂	000 = 0, 100 = 4, 111 = 7	47 ₈	0010 = 2, 0111 = 7	0x27
10110010 ₂	010 = 2, 110 = 6, 010 = 2	262 ₈	1011 = 11, 0010 = 2	0xB2
11011001 ₂	011 = 3, 011 = 3, 001 = 1	331 ₈	1101 = 13, 1001 = 9	0xD9

Digital circuits can only manipulate representations of numbers in binary with a finite number of bits. This limitation means that additionally to a technique to translate numbers to binary, we must also decide the size of such encoding and what happens when such size is not enough.

Size of encoding(=number of bits):

- n bits could represent number from 0 to 2^n-1 in decimal. **e.g.** 7 bits with interval [0, 127].
- The last number of such interval will be encoded as all 1. **e.g.** 1111111=127

Overflow is the situation that the result out of interval. **e.g.** in this case, representing 128 will cause an overflow.

The number of bits used to encode natural numbers changes from circuit to circuit and is one of the parameters that defines what is called its **architecture**.

*Other interesting facts:

“Any binary number terminated in three zeros is multiple of 8.”

“Any octal number terminated in three zeros is multiple of 512.”

Encoding integers: Sign & Magnitude, 2's complement

Schemes	Sign & Magnitude	2's complement
Steps(Positive)	Convert to binary	Convert to binary
Steps(Negative number)	1. Convert to binary 2. Add 1 for sign at the beginning.	1. Convert to binary 2. Flip all bits 3. Add 1

Schemes	Sign & Magnitude	2's complement
Example (-23 in 8 bits)	1. To binary: 00010111 2. Add sign: 1 0010111 1 0010111 = -23	1. To binary: 00010111 2. Flip all bits: 11101000 3. Add 1: 11101001 = -23
Minimum number	Start with 1 followed by all 1.	Start with 1 followed by all 0.
Example(8 bits)	1 1111111 = -127	10000000 = -128
Maximum number	Start with 0 followed by all 1.	Start with 0 followed by all 1.
Example(8 bits)	0 1111111 = 127	01111111 = 127
Interval(n bits)	$[-(2^{n-1}-1), 2^{n-1}-1]$	$[-(2^{n-1}), 2^{n-1}-1]$
Example(8 bits)	[-127, 127]	[-128, 127]
Encoding 0 (8 bits)	Two zero: 00000000 OR 10000000	One zero: 00000000
Encoding 1/-1 (8 bits)	0 0000001 = 1 1 0000001 = -1	00000001 = 1 11111111 = -1
Back To integer	1. Remove first bit for sign 2. Convert binary to integer 3. Add minus- if sign is 1	If the most significant bit is 0(positive), Binary to decimal. If the most significant bit is 1(negative) 1. Flip all bits 2. Add 1 3. Convert binary to decimal and add minus-.
Bit Extension: e.g. From 8 bits to 10 bits -23 for Example:	Add zeros in the most significant bits of the magnitude. Maintaining the sign. 1 0010111 → 1 000010111 = -23	Positive: Add zeros to the most significant bits until the new length is reached. Negative: Add ones to the most significant bits. 11101001 → 1111101001 = -23

*Convert 2's complement to integer, we can use another way(Again, -23 for **example**, its 2's complement representation is N=11101001, it is encoded in n=8 bits):

$$ABS(N) - 2^n = ABS(11101001) - 2^8 = 233 - 256 = -23$$

Advantages for 2's complement:

- Zero has a unique representation and it corresponds with its value in base 2.
- The most significant bit (or the left most) still represents the sign of the number with identical correspondence as the one used in sign & magnitude. 0 for positive, 1 for negative.
- The positive integers are encoded exactly as the natural numbers, with their encoding in base 2.
- The addition and subtraction operations can be done following base 2 rules as if the numbers were naturals.

Floating Point

Suppose that two real numbers are added, and the result does not correspond with one of the values included in the encoding. This situation is perfectly possible because only a subset of possible values are encoded. The only possible solution in digital circuits is to **approximate** the result by the closet number that can be represented.

This means that when a digital circuit manipulates real numbers it introduces **an error**. In principle any real number that is manipulated by a digital circuit may contain some error, which in some cases could be zero.

Real numbers are encoded in binary logic with the technique known as

$$\begin{aligned}
 101.111 &= 10.1111 * 2^1 \\
 &= 1.01111 * 2^2 \\
 &= 0.101111 * 2^3
 \end{aligned}$$

floating point.

The technique to achieve the proper encoding is to encode separately with 3 ingredients: the **sign**, the **mantissa**, and the **exponent** (as the base is implicitly 2).

Every real number is represented as a set of digits to the left of the point and another set of digits to the right of the point. e.g. 111.0111101. The part to the right of the point (0111101) is called **mantissa** (or significand).

$$\begin{aligned} 101.111 &= 2^2 + 2^0 + 2^{-1} + 2^{-2} + 2^{-3} \\ &= 4 + 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} \\ &= 5.875 \end{aligned}$$

Multiplying and dividing by the base power **exponent** allows the point to be moved to the left and right respectively.

The advantage of the floating point representation is that it allows to represent certain very big and very small numbers in a very compact form.

Sign	Mantissa	Exponent
------	----------	----------

Structure of the floating point representation of a real number

In this type of encoding, we need to know:

- The overall size of encoding.
- How many bits within the encoding are devoted to the mantissa.
- How many of them to the exponent.
- How is the exponent value being encoded (sign and magnitude, 2s complement, 1s complement, excess K, etc.)

The following example assumes that exponent is encoded in 2s complement.

How do we encode 0?

The way zero is encoded is typically assigning a special combination of bits in the mantissa and exponent and check for the presence of such combination before performing such operation.

Interval of floating point representation

Numbers	Negative numbers	Positive numbers
x bits for mantissa n bits for exponent	$[-0.1111... * (2^{n-1} - 1), -0.1 * 2^{-(n-1)}]$ There are x ones.	$[0.1 * 2^{-(n-1)}, 0.1111... * (2^{n-1} - 1)]$ There are x ones.
Exponent	biggest $(2^{n-1} - 1)$, smallest $2^{-(n-1)}$	smallest $2^{-(n-1)}$, biggest $(2^{n-1} - 1)$
Example1: x=7, n=6 Higher precision	$[-0.1111111 * 2^{31}, -0.1 * 2^{-32}]$	$[0.1 * 2^{-32}, 0.1111111 * 2^{31}]$
Example2: x=5, n=8 Higher accuracy	$[-0.11111 * 2^{127}, -0.1 * 2^{-128}]$	$[0.1 * 2^{-128}, 0.11111 * 2^{127}]$

Two floating point encoding schemes with identical overall size may have different precision due to the distribution of the bits between the **mantissa** and the **exponent**.

Encoding real number Steps (8bits for mantissa, 6 bits for exponent)	Example1 (-101.111=-5.875)	Example2 (0.0111=0.375)
1. Add sign for the first bit.	1 XXXXXXXX XXXXXX	0 XXXXXXXX XXXXXX
2. Obtain mantissa by multiplying or dividing by 2 so that the first non-zero digit is at the right of the point.	101.111 = 0.101111 * 2 ³ Mantissa: 101111 1 00101111 XXXXXX	0.0111 = 0.111 * 2 ⁻¹ Mantissa: 111 0 0000111 XXXXXX
3. Obtain exponent by encoding in 2s complement.	3 → 000011 (in 6 bits in 2's complement) 1 00101111 000011	-1 → 111111 (in 6 bits in 2's complement) 0 00001111 111111

Back to real number(8 bits for mantissa, 6 bits for exponent)	Example1 (1 00101111 000011)	Example2 (0 00000111 111111)
1. Add minus- if first bit is 1	-	
2. Write mantissa, ignore first zeros	-0.101111	0.111
3. Convert exponent back to decimal (Note it is represented in 2s complement)	000011 = 3	111111 = -1
4. Multiply mantissa by 2^{exponent}	$-0.101111 * 2^3 = -101.111$	$0.111 * 2^{-1} = 0.01111$

Precision is a property of a generic encoding. In other words, we may compare the precision provided by using 10 or 12 bits for the mantissa. **Bigger mantissa, higher precision.**

Accuracy refers to how close a binary encoding of a real number is to the real magnitude we want to encode. **Bigger exponent, higher interval(accuracy)**

Overflow: Out of interval

The way the microprocessors deal with it is simply to detect it and raise an exception so that it can be treated by some specific code.

Underflow: Below smallest positive number/biggest negative number.

Approximating a non-zero value by zero has a devastating effect in some operations. Suppose the result of this operation is then multiplied by a very large number, in that case, the result is zero, and therefore the error is enormous.

From the point of view of a computer system, the designers must include the required hardware to detect these situations when performing mathematical calculations.

*The Institute of Electrical and Electronics Engineers (known simply as the IEEE) has defined a standard to encoded floating point numbers known as the IEEE 754 format. The standard defines five representations: binary floating point with 32, 64 and 128 bits, and decimal floating point with 64 and 128 bits.

Encode set of symbols

To encode symbols, we must know

- How many bits shall we use according to the number of elements. See Binary Logic.
- Correspondence(what number represents what symbol)

1. Encode Characters

- **ASCII**, original 7 bits, extended to 8bits=256 symbols.

*The first 32 combinations (from 00000000 or 0x00, to 00011111 or 0x1F) are used to encode symbols that are not printable and were used to handle the transmission of text between telegraphs.

*The symbols with codes from 32 (0x20) to 127 (0x7F) correspond with printable symbols. For example, the white space is encoded as 0x20.

*Letters “a” to “z” occupy the codes 0x61 to 0x7A. Its corresponding uppercase symbols are in the codes 0x41 to 0x5A (the difference between upper and lowercase codes is a constant).

If you want to transform a digit in its ASCII encoding, you should add 48.

If you want to transform a character in ASCII to its uppercase, subtract 32.

Limitation: Can't encode à ü...

- **Unicode**, has been adopted by most of the hardware and software companies.
 - UTF-8 8bits compatible with ASCII,
 - UTF-16 16bits, more characters. Usually represent in 0xFFFF format.
 - UTF-32 32bits

2. Encode Instructions

- UAL-1: an operation and two operands that are integers in the range [0,255]

Operation(2 bits)

add, sub, mul, div

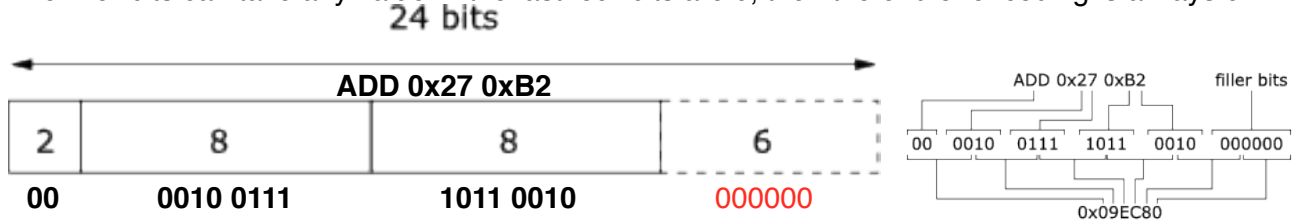
00, 01, 10, 11

In this scheme, we have

$2^2 * 2^8 * 2^8 = 2^{18}$ 18 bits(262144 combinations)

We will use an encoding with size that is multiple of 8 so we will use 24 bits (3 bytes) by adding 0 in the end. These 6 bits are all 0 as **filler bits**.

The filler bits can take any value. If the last four bits are 0, then the end of encoding is always 0.



ADD 0x27 0xB2 encoded in UAL-1 format is 0x09EC80.

The most important property of this encoding is that the translation of any instruction to its representation in binary and vice versa is a systematic process, and therefore can be done automatically by a computer system.

ELEC 1601 Week 3

Computer Memory

Navigation

- Computer memory
- Memory Operations
- Connection between memory and processor
- Data storage
- Arrays(tables) in memory
- Indirection

Computer memory

RAM(random access memory, or memory in general) is simply a large table of cells, each of them capable of storing a pre-defined unit of information, for example one byte.

Internally, each memory cell is implemented using a set of **transistors** and gates that are capable of remembering a previously defined set of bits.

When we describe a memory, we usually have:

A. **The width(size) of the cell**(bits), the most common is 1 byte.

The way data is accessed in memory is determined by the size of its cells.

For example, if the cells store 1 byte each, the data is accessed 1 byte at a time.

Some computer systems are capable of accessing a set of contiguous cells in one single operation, but this is simply to optimize the access time. Still, each cell is referred by its unique address. So, if you want to change the value of a single bit in a byte of a memory storing 1 byte per cell, you still need to read the entire byte, modify the single bit, and write the byte back again to memory.

B. **Number of cells**, starts from 0.

The number of a cell is called its **memory address** (a natural number encoded in base 2). It is usually prefixed by the symbol “@”.

C. Number of bits for the address

All the address values of a memory is usually called its **address space**.

1 byte	
0x20	0x21
0x21	0x2B
0x22	0x4D
0x23	0xFE
0x24	0xD2
0x25	0x55
0x26	0x27
0x27	0x1C
0x28	0xB3
0x29	0x9A

Two types of volatile memory SRAM vs DRAM

Type of volatile memory	Static RAM(SRAM)	Dynamic RAM(DRAM)
Common	Volatile: Data is stored in a cell until it is overwritten or the power is stopped. Supports the same operations(read and write) and organized in cells. They both are used by computer systems.	
Hardware(Store value in)	Transistors	Capacitors
Working Principle	The number of transistors required to implement one cell is around 100.	If one of these capacitors is charged, it is storing a one, and if discharged it is storing a zero.
Time to perform read operation	10 nanoseconds(10^{-8} second)	Cells take a few milliseconds(ms, 10^{-3} second) to loose their charge.
Structure of cells	Complex	Simpler
Advantages	Faster for operations.	*Higher capacity(number of bytes). Low cost.
Disadvantage	Lower capacity. High cost.	Forgets data. Slower for operations.
Preference	For components that need to be extremely fast.	For components where a large amount of memory is needed.

*The size of a cell in a **dynamic** memory is approximately 1/4 of a **static** cell, and it is much simpler to design and produce.

The problem for DRAM is that capacitors leak current, and therefore, those that are charged (and storing a one) start to discharge over certain time interval. If no operation is done, and even if the chip has power, the charge in the capacitors disappears and the cell loses its value. However, if a read operation is done over a cell, it refreshes the charges of those capacitors storing ones.

This problem is alleviated by including in the chip itself a mechanism to refresh the cells by simply reading all of them continuously before the capacitors lose their charge. Each memory cell must be read after certain time interval, thus *slowing down* the overall read/write operations. This is the reason why this memory is called **dynamic** because it is constantly being read or refreshed.

Memory Operations

The two **operations** allowed in memory are **reading a value** and **writing a value**.

In a read operation, the memory receives an address (a set of bits encoding a memory address) and returns the content (set of bits stored in) in the cell.

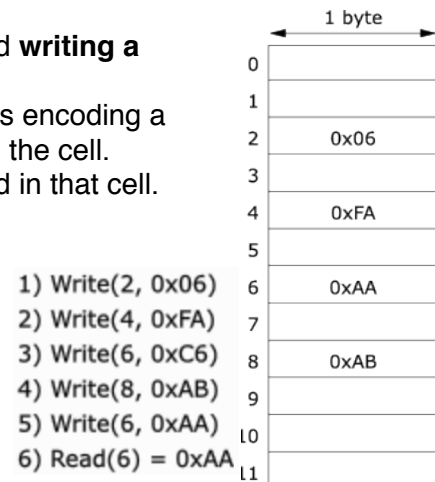
Byte Read(Address a): Given an address, returns the byte stored in that cell.

In the write operation, the memory receives an address and a value (2 sets of bits), writes the value in the cell with the given address, and no result is produced.

void Write(Address a, Byte b): Store byte b in the cell with address a.

The data stored initially in memory before any read or write operation is executed is **undefined**.

If the first operation executed in RAM memory after starting a computer system is a read operation, the result is **undefined** or **garbage**. Thus, every read operation must be performed on a cell in which content has been previously written.



Always start with a write operation

As memories are digital circuits, all the **data** must be encoded in base 2, and this includes the data need for the read and write **operations**. **Addresses** also need to be encoded in base 2.

Every cell in memory has an address, and therefore, the number of cells corresponds to the maximum value allowed for an address + 1.

A memory chip with addresses represented by n bits, cannot have more than 2^n cells numbered from 0 to $2^n - 1$. i.e. the size S of a memory chip and the number of bits n used to encode the addresses must satisfy:

$$S \leq 2^n$$

The number of cells in a memory chip is usually a power of two.

Memory Size = numbers of cells * cell size

Cell size is typically 1 byte minimum.

8 bits = 1 byte.

4 Gigabytes = 2^{32} bytes, so 32 bits are required to encode address.

Thus 32 bits (4 bytes) are used to store memory address.

Prefix	Symbol	Size(Power)
kilo	K	2^{10} bytes
mega	M	2^{20} bytes
giga	G	2^{30} bytes
tera	T	2^{40} bytes

Connection between memory and processor

A **bus** is a set of wires in which several circuits can read and write binary values.

The **address bus** is used to send the address of the operation to the memory chip.

The **data bus** is used to send the data from the processor to the memory chip when write or from the memory chip to the processor when read.

An additional one(**R/W**) bit

wire is needed so that the processor notifies the memory chip the type of operation.

1 for read, 0 for write.

Size of Java basic data types¶

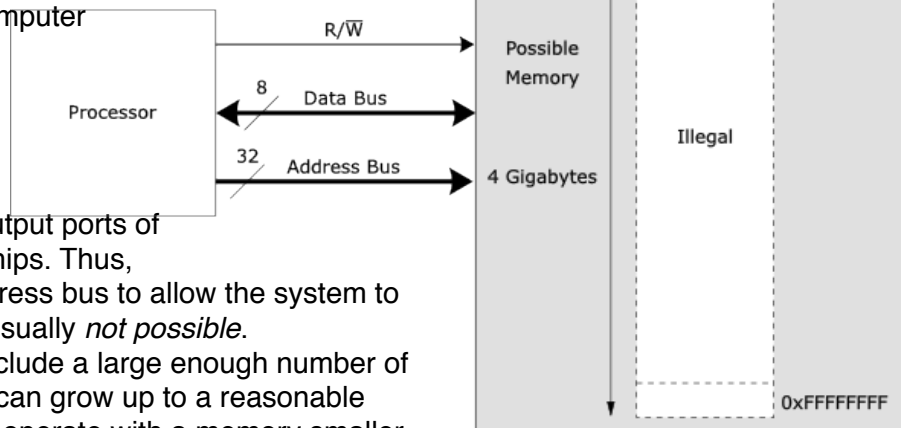
Type	Values	Size	Range
boolean	true, false	1 bit	
byte	Integer	8 bits	[-128, 127]
char	Character (Unicode)	16 bits	[0, 65535]
short	Integer	16 bits	[-32768, 32767]
int	Integer	32 bits	[-2147483648, 2147483647]
long	Integer	64 bits	[-9223372036854775808, 9223372036854775807]
float	IEEE-754 Floating point	32 bits	[±1.4012985E-45, ±3.4028235E+38]
double	IEEE-754 Floating point	64 bits	[±4.94065645841246544E-324, ±1.7976931348623157E+308]

The *slanted line* is the notation used to denote the number of bits or wires that are part of that signal.

e.g. 8 bits for data bus in the figure, address bus has 32 bits. From the relation between the number of bits used to encode the address we can deduce that the maximum memory for this computer system is 2^{32} bytes.

Buses are implemented as metallic tracks in a chip surface. The wires are connected to the input and output ports of the processor and memory chips. Thus, adding an extra bit to the address bus to allow the system to have double the memory, is usually *not possible*. Today's computer systems include a large enough number of address bits so that memory can grow up to a reasonable size. Computer systems may operate with a memory smaller than what is allowed by the number of bits used to encode the address.

The figure shows the situation in which a processor has 32 bits to encode the address, and yet, the memory physically available is only 256 Megabytes.

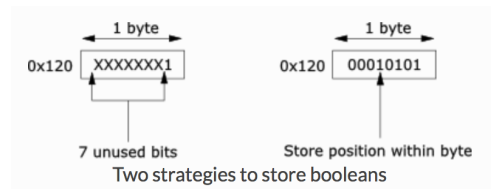


Processors typically include a mechanism by which the upper limit of the address space is known. Before any operation is executed in memory, the address is checked against this limit. If the address is beyond this value, the operation is stopped and an exception is raised.

Data Storage

Processors are typically designed to execute certain operations very fast over data of certain sizes. High level programming languages such as **Java** define a set of data structures called *basic* plus a mechanism to combine them and create more complex **data structures**.

In those languages that are compiled, the **compiler** is in charge of mapping the complex data structures to the memory of the computer system. If the processor is not capable of performing certain operations (for example, multiplication of numbers of 128 bits), it typically resorts to some library that can execute the operation at a much higher cost in time.



The simple rule to store data in memory is to use as many *consecutive* cells as needed to store a complex data structure. The address of the first cell from which the data structure is stored will be referred to as the **data address**. Analogously, when a data structure is stored at a certain memory address, what it really means is that it is stored with as many memory cells as required starting at the given address.

Storage for different basic data types:

1. Booleans

- Eight boolean values can be stored in a 1 byte memory cell. However, the disadvantage of this technique is that to access the boolean, we need to know the memory address and the position of the bit inside the byte, which is a number between 0 and 7.
- One boolean variable can be stored in an entire memory cell leaving the rest of bits untouched. In this case, all bits but one are wasted, but the access to the value is much faster.

2. Characters

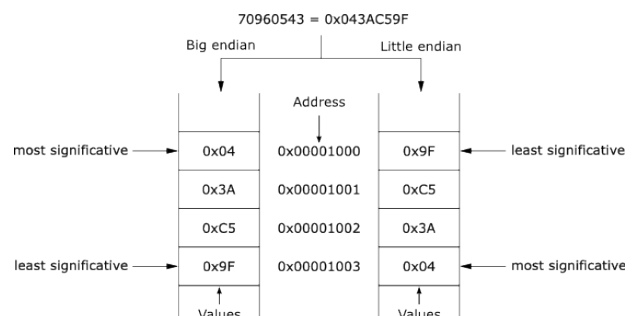
The storage depends on the encoding scheme that is used.

- ASCII—8 bits are required to encode each character. In this case, if the size of the memory cells is one byte, one single cell is needed per character. A sequence of characters will then occupy consecutive cells in memory. The figure on the right shows how the sequence of characters “My first string.” encoded in ASCII is stored in memory.
- Unicode UTF-16—Each symbol would occupy two consecutive memory cells instead of one.

Address	Values	
0x120	0x4D	"M"
0x121	0x79	"y"
0x122	0x20	" "
0x123	0x66	"f"
0x124	0x69	"i"
0x125	0x72	"r"
0x126	0x73	"s"
0x127	0x74	"t"
0x128	0x20	" "
0x129	0x73	"s"
0x12A	0x74	"t"
0x12B	0x72	"r"
0x12C	0x69	"i"
0x12D	0x6E	"n"
0x12E	0x67	"g"
0x12F	0x2E	"."

3. Integers and natural numbers

It is essential to know in advance the size of the representation. The most common sizes for these numbers are 2, 4, 8 or even 16 bytes. The size of these numbers is decided based on the circuits needed for their operations. For example, if an integer size of 32 bits (4 bytes) is chosen, it is likely that the processor has a circuit to perform addition and subtraction of numbers of this size.



We also need to know the order in which the bytes are arranged. They can be stored from least significant to most significant, or vice versa. These two possibilities are equally used by computer system designers and as a consequence today’s processors are divided along this feature.

These two possibilities to store integers and naturals bigger than one byte are known as **little endian** (when the first byte stored is the least significant) and **big endian** (when the first byte stored is the most significant). Each computer system uses one of the two methods to store and manipulate all the integers and natural numbers.

- Integers: 0x3F2A (positive because 3 = 0011 with first letter 0)
 - 0x3F big endian $\longleftrightarrow$ 0x2A little endian
 - 0x2A big endian $\longleftrightarrow$ 0x3F little endian

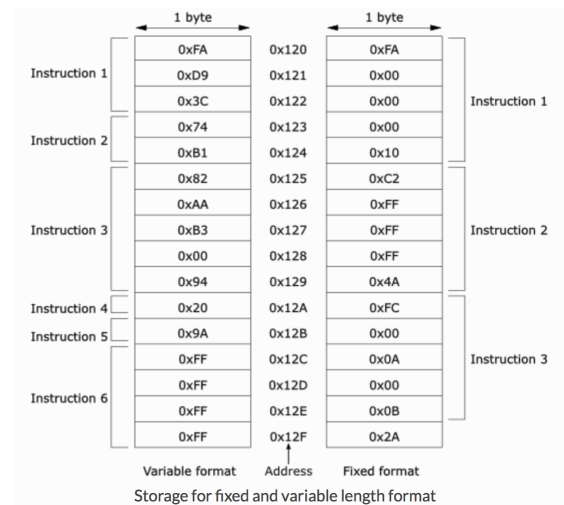
The problem with this duality appears when there are two processors in a system with different policies for this storage. In this situation, manipulating integers and naturals may require additional operations to swap the order in which the bytes are manipulated.

There are numerous reasons in favor and against each policies, but none of them are enough to rule one better than the other. In the case of little endian, if the size of the representation needs to be extended, we only need to use extra memory cells at the end. If the number is stored using big endian, the bytes also need to be re-arranged.

Note: The little/big endian policy only applies when the data is stored in memory.

4. Machine instructions

- Using **fixed instruction length** to access instructions is very easy as the next instruction is a fixed number of bytes away from the previous one.
 - Using **variable instruction length**, the address of the next instruction in a sequence needs to be deducted from the address of the current instruction and its size.
- First obtain one instruction from memory at a specific address.
 - Interprets its content, deduces its size, and this size is then added to the address of the instruction to obtain the address of the following instruction in the sequence.



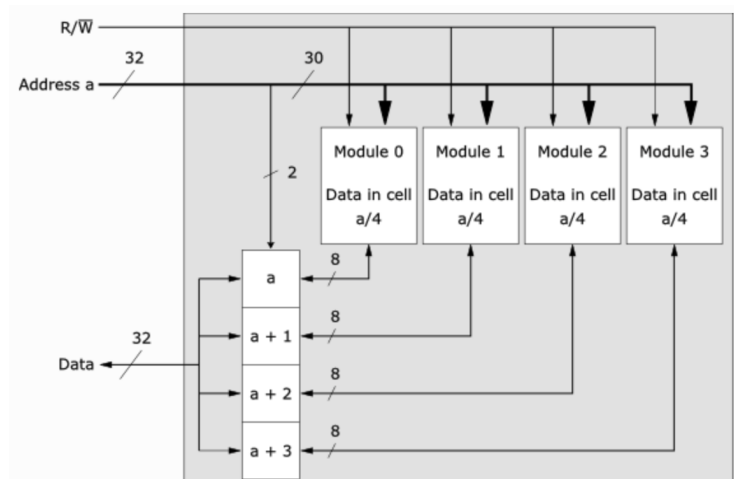
When storing instructions there is no need to make the distinction between little and big endian because in this encoding there is not such thing as a most significant byte in the instruction encoding. The policy used when writing the binary encoding of one instruction is to write the bytes in the order in which they are stored in memory.

• *Read and write operations in memory

The time it takes to read or write a value is much larger than the speed of the processor. In other words, every time an operation is required from memory, the overall system is delayed waiting for the memory chip to finish it becoming a bottleneck for the overall system performance.

Designers have come up with a huge number of design decisions and techniques included in the architecture of a processor that reduce this difference.

- Perform read and write operations for several cells at the same time. The memory still maintains its structure and the possibility of addressing each cell individually, however, some systems allow the read and write operations to be done in blocks of consecutive memory cells. This technique based on the observation that when reading data from memory, most of the time several consecutive bytes are needed (think, for example, when reading an integer encoded with 4 bytes). The drawback of this technique is that there might be some operations that manipulate more cells than what is really required.
- Most systems allow the memory operations to be performed in one single step for more than one byte. This can be done by combining multiple memory chips or modules to behave as a regular memory. **For example**, let us consider the case in which we want a memory with 1 byte cells, 4 Gigabyte size (therefore with addresses encoded with 32 bits), and that allows operations over four bytes simultaneously. The solution consists on using four memory chips connected in parallel, so each takes care of the operation for one of the four bytes. The resulting memory has the cell with address 0 stored in the first chip, the address 1 in the second chip, third address in the third chip, fourth address in the fourth chip, fifth address again in the first chip and so on.



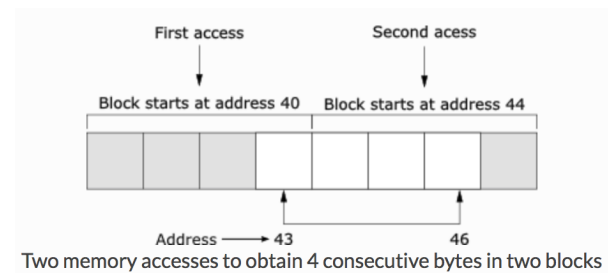
Access to memory in blocks of four bytes

With this scheme, and if we number the

modules from 0 to 3, the cell with memory address in position p is stored in the module number $p \% 4$.

The only requirement for this scheme to work is that the blocks of four cells have to be aligned with memory addresses that are multiple of 4. In other words, the cells in address 0, 1, 2 and 3 can be returned in the time it takes one module to perform the operation. This does not apply if we require the addresses 2, 3, 4 and 5, because, although they are in consecutive addresses, they are not part of the same block of four cells.

Each memory module receives the most significant 30 bits of the 32 address bits. This is because the operation is performed over a block of four consecutive cells. The address is obtained then by dropping the two least significant bits of the address. Each module then provides one of the byte in the group. With this configuration four bytes are read or written in the time it takes one module to execute the operation, as all four of them work in parallel. With this technique, even though internally there are four memory blocks, the complete circuit behaves as a 4 Gigabyte, 1 byte per cell, regular memory.



What happens if the processor wants to access to 4 consecutive bytes with an address that is not a multiple of four?

This type of memory requests are called unaligned memory operations, and requires the memory to perform more than one manipulate the requested information.

The memory would then access the first block (with address 40), take only the last byte, and then access the block in address 44 and discard the last byte (with address 47).

Arrays(tables) in memory

If an array contains n elements, and each element requires m bytes. If the elements in the table are stored starting from the generic address a . The address of the element index would be:

$$address(table[index]) = a + (m * index)$$

Note: If a program *does not check* that the index is correct when accessing an array of 25 integers, when accessing the table[25], since no check is done, the program reads an incorrect memory position.

There are two techniques to know the number of elements in a table when stored in memory.

1. Placing a special value which is not used for any other purpose but to mark the end of the table. When encoding characters, there is one extra symbol that is included in the encoding scheme, and is used to mark the end of a sequence.
 - But this technique only works when there is a special code for the end of sequence. It does not work for the case of integers or naturals if they are encoded using base 2 or 2's complement. In both encoding schemes all values are valid numbers, and therefore there is no spare encoding for the end of sequence.
2. Store the size of the array in another memory location as an additional natural number.
 - With this solution, if the table needs to be traversed, the number of elements must be first obtained from this memory location, and then used it to stop the traversal at the right location.

- Arrays in java(See Also INFO 1103 Week 5)

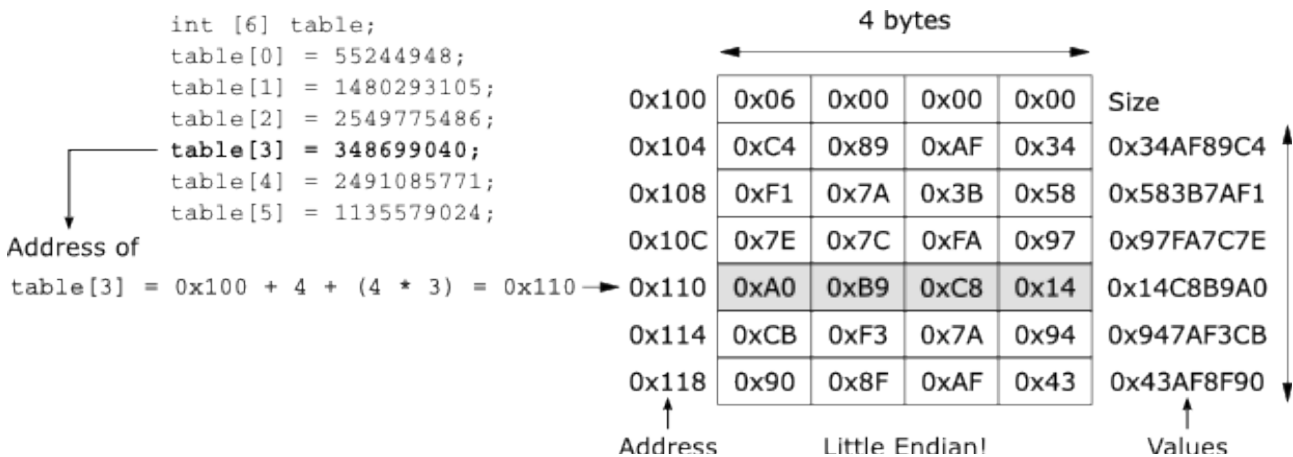
The technique used by Java to implement this mechanism is to store the size of the table in the first memory positions of the table. After this number, all the elements of the table are then stored in consecutive positions.

Before accessing the element, the program executes some additional instructions to verify that the previous equation(integer) is satisfied for the value of index.

Java stores the size of the array in 4 bytes.

In a Java program, before accessing an element in position i in a table in which each element occupy t bytes, and first element starts at position d , the program performs following operations:

1. Obtain integer stored in position d .
2. Check that $0 \leq i$, if not satisfied, raise exception `ArrayIndexOutOfBoundsException`.
3. Check that $i < \text{size}$, if not satisfied, raise exception `ArrayIndexOutOfBoundsException`.
4. Calculate the address of the element as $d + 4 + (t * i)$



Indirection

We can store in memory the binary representation of **a memory address** in four consecutive bytes.

Note that In different computer system, memory addresses, when stored in memory (with an indirection) may not occupy 4 bytes.

From this point of view, a memory address can be considered either as the unique identifier of a memory cell, but also as a natural number that can be manipulated normally.

Indirection: memory address contains the memory address of another value.

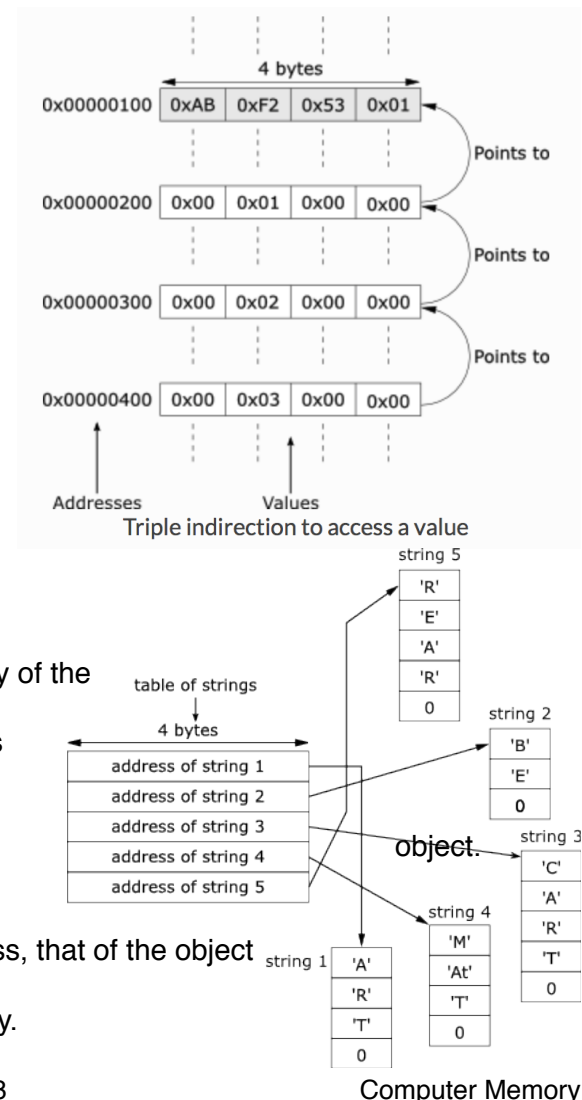
The **indirection** technique can be chained an arbitrary number of times. The address of the position that contains the address of the data can itself be stored in memory.

Indirection Examples

1. Storing a table of strings
 - Create a table of the addresses of strings
 - Sorting the strings can be done without moving any of the characters in memory.
 - The resulting table contains in each of its elements address of a string.
2. References in Java
 - Java uses indirection to access any element of an

In Java, objects are manipulated using references. Internally, Java programs represent references as memory locations that contain another memory address, that of the object they refer to.

The object is created at an arbitrary position in memory.

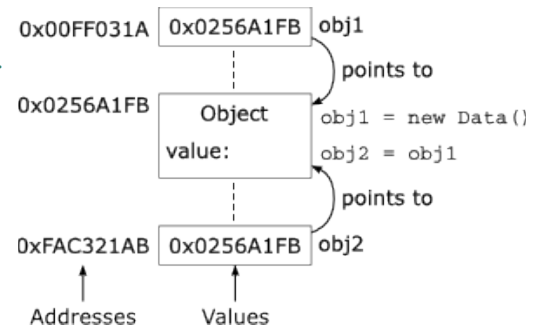


Example1: Indirection with the same object

```

3 public class Data {
4     int value;
5     public static void main(String[] args) {
6         Data obj1, obj2; //obj1, obj2 are not objects, but references.
7
8         obj1 = new Data(); //only one object is created
9         obj1.value = 1; //obj1 is a reference pointing to new object
10
11        obj2 = obj1; //assign obj2 the same reference as obj1
12
13        obj2.value = 2; //obj2 only stores address of the object
14
15        System.out.println(obj1.value);
16
17        return;
18    }
19 }
20 }

```



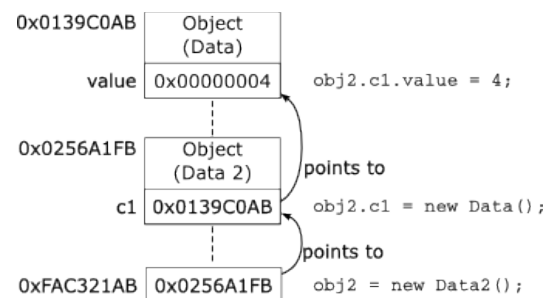
In two additional memory locations the references obj1 and obj2 are stored. The first one receives its value after executing the call to the method **new**. The second reference gets the same value that is stored in obj1. From that point on, any changes to the object done through obj2 is visible through obj1. A second indirection is then executed to access the field in that object and assign the value. i.e. obj1, obj2 refers to the same object.

Example2: Double indirection with reference in Java

```

3 public class Data2 {
4     Data c1;
5     public static void main(String[] args) {
6         Data2 obj2;
7
8         obj2 = new Data2(); //point to an object of class Data2
9         obj2.c1 = new Data(); //another reference to object of Data
10
11        obj2.c1.value = 4; //the value of object Data is 4
12
13        return;
14    }
15 }
16 }

```



A class in Java may contain as one of its fields references to other objects.

In this case, the reference obj2 points to an object of class Data2 which itself contains a reference to an object of class Data. The execution of the last line of code in the main method requires a double indirection. The value of the variable obj2 is the address of the object of class Data2, and once it is accessed, we need to extract its field c1. The value of this field is another reference and therefore contains the address of the object of class Data. A second indirection is then executed to access the field in that object and assign the value.

ELEC 1601 Week 4

Boolean Logic and Combinational Circuits

Navigation

- Boolean Algebra
- Logic Gates
- Design Criteria for Digital Circuits
- Example of combinational circuits

Boolean Algebra

A **Boolean Algebra** has only 2 values, 0 and 1 (represents low & high, off & on in circuits)

Three operations of Boolean Algebra:

1. Conjunction(and): $a * b$ (or ab). **e.g.** $ab = 1$ when all the symbols are 1.
2. Disjunction(or): $a + b$. **e.g.** $a + b = 1$ when any of the symbol is 1. $1 + 1 = 1$
3. Negation(not): a' . Opposite value of a symbol.

Any symbol in Boolean algebra can only take the values true and false. Thus, we can easily enumerate all possible values of a simple expression.

Truth table is a table contains all the possible values of symbols and all the possible result of a certain boolean expression. If given n symbols, the truth table will have:

- 2^n rows
- n+p column (additional p column for results, result can be more than 1, for multi-output)

The process of substituting the symbols in an expression by a specific set of values is called **evaluation**. The last column of the truth table of a Boolean expression is then obtained by performing as many evaluations as possible combinations of values for its symbols.

x	y	z	Output
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

If the truth table of different boolean expressions are totally identical, the expressions are **equivalent**. In other words, the two expressions are equivalent if for every possible combination of values of its symbols, they produce the same results, or have identical truth table.

Boolean Properties: $(x' + y') = xy$; $(x + y) = x'y'$; $(x'y')' = x + y$; $(xy)' = x' + y'$

Identity	$1 * x = x, 0 + x = x$
Null	$0 * x = 0, 1 + x = 1$
Idempotent	$x * x = x, x + x = x$
Inverse	$x * x' = 0, x + x' = 1$
Commutative	$x * y = y * x, x + y = y + x$
Redundancy	$x + x'y = x + y, x(x' + y) = xy$
Associative	$(xy)z = x(yz), (x + y) + z = x + (y + z)$
Distributive	$x + yz = (x + y)(x + z), x(y + z) = xy + xz$
Absortion	$x(x + y) = x, x + xy = x$
De Morgan	$(xy)' = x' + y', (x + y)' = x'y'$
Double Complement	$(x')' = x$

← Properties

Properties can be used to write Boolean expressions in ways that can be simplified.

$$\begin{aligned}
 &(x + y)(x + y')(xz')' \\
 &= (x + y)(x + y')(x' + z) \quad (\text{De Morgan}) \\
 &= (xx + xy' + xy + yy')(x' + z) \quad (\text{Distributive}) \\
 &= (x + xy' + xy + 0)(x' + z) \quad (\text{Idempotent, Inverse}) \\
 &= (x + xy' + xy)(x' + z) \quad (\text{Identity}) \\
 &= x(x' + z) \quad (\text{Absortion}) \\
 &= xx' + xz \quad (\text{Distributive}) \\
 &= xz \quad (\text{Inverse, Identity})
 \end{aligned}$$

This simplification process has transformed a three symbols (x, y, z) Boolean expression into a simpler expression with only two symbols (x, z). This case, y is said to be a **redundant variable**.

We usually choose the most compact expression: *less space, faster*.

Canonical representations

Given a truth table, and work out the boolean expression:

Step	Sum of Product(SOP)	Example
1	Select rows that have result = 1	(0, 1, 1, 1) (1, 1, 0, 1) (1, 1, 1, 1)
2	Write Product of symbols, and negate value 0 symbol.	(0, 1, 1, 1) $a'bc$ (1, 1, 0, 1) abc' (1, 1, 1, 1) abc
3	Add+ together in a disjunction.	$a'bc+abc'+abc$
Step	Products of Sum(POS)	Example
1	Select rows that have result = 0	(0, 0, 0, 0), (0, 0, 1, 0), (0, 1, 0, 0), (1, 0, 0, 0), (1, 0, 1, 0)
2	Write Sum of symbols, and negate value 1 symbol.	(0, 0, 0, 0) $a+b+c$ (0, 0, 1, 0) $a+b+c'$ (0, 1, 0, 0) $a+b'+c$ (1, 0, 0, 0) $a'+b+c$ (1, 0, 1, 0) $a'+b+c'$
3	Multiple* together in a conjunction.	$(a+b+c)(a+b+c')(a+b'+c)$ $(a'+b+c)(a'+b+c')$

a	b	c	Result
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Example Truth Table

Boolean functions can be easily extended to have more than one result to what is known as a **Multi-output Boolean Function**.

Logic Gates

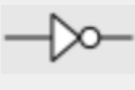
All digital circuits use as building blocks units that manipulate bits following Boolean algebraic rules. The reason is that current manufacturing procedures are capable of creating, using a small number of devices called **transistors**, small circuits that detect only two possible values (high and low), and implement the operations of conjunction, disjunction and negation as described in Boolean algebra.

Even though each of these small circuits implement a simple operation, the combination of millions of them allow digital circuits to perform highly sophisticated operations at very high speeds.

At its basic level, digital circuits are only capable of detecting two values, or two levels of voltage: high and low. Thus, we decide to encode true as high, or 1, and false as low or zero.

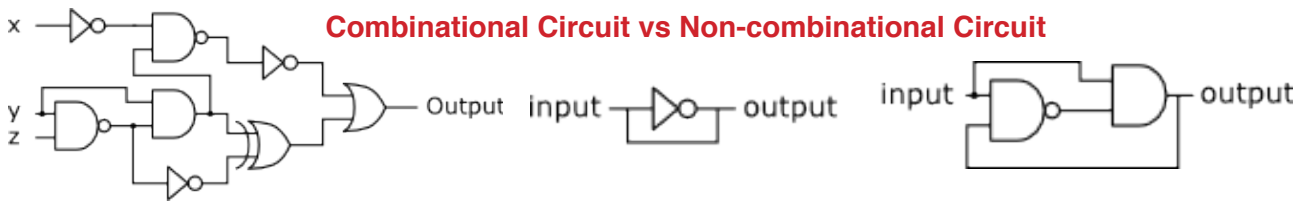
The circuits that implement Boolean algebra operations are commonly known as **gates**.

Each gate behaves as the operations in the Boolean algebra previously described. Given the values of inputs, they almost instantaneously produce the output value according to the operation. The truth table for the gates is identical to the truth table of the corresponding Boolean expressions.

Gate	AND	OR	Inverter	NAND	NOR	XOR	XNOR
Condition	all 1	any 1		all 1	any 1	Odd 1	Even 1
Output	1	1	x'	0	0	1	1
Explain	$x * y$	$x + y$		AND+Inverter($x * y$)'	OR+Inverter($x + y$)'	$(xy')+(x'y)$	$(xy)+(x'y')$
Figure							

The number of inputs in these basic gates can also be extended to more than two. The same rules apply to obtain the value of the output. In reality, a gate with a large number of inputs requires a complex circuit and they are typically implemented using a combination of smaller gates.

A combinational circuit only depends on the value of its inputs. Non-combinational vice versa.



The truth table of a combinational circuit is obtained similar to the case of a Boolean expression. For a circuit with n inputs, the corresponding truth table will have 2^n rows. The columns correspond with each of the inputs, and the last column is the output.

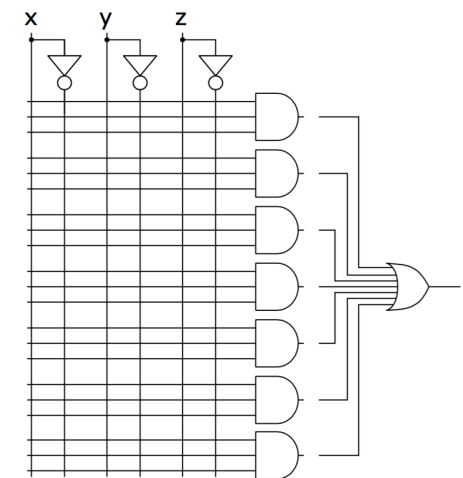
From a Combinational Circuit to a Boolean Function:

1. Write truth table.
2. Get SOP or POS.

From a Boolean Function to a Combinational Circuit:

1. Write truth table.
2. Get SOP.
3. Apply in the right figure. —>

Alternatively, we can Get POS, and change all AND Gate to OR Gate, the last OR Gate to AND Gate.



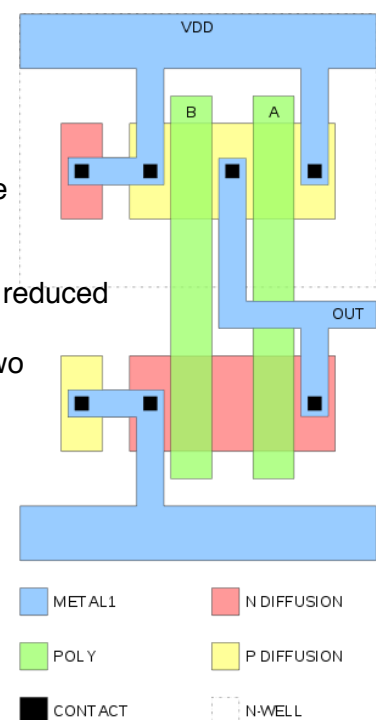
Design Criteria for Digital Circuits

Nowadays, the process to implement logic gates uses **silicon**, which is a semi-conductor component. Using different particles that are etched in a surface and applying certain voltages, a small area can either be isolated (no current goes across) or connected (a current traverses it). This technique is the basis to create transistors, and logic gates can be implemented connecting various transistors.

As you can see, four types of components are used: metal (which can be laid out in various layers), poly, n-diffusion, and p-diffusion. The size of these connections determine the space occupied by a gate and the number of gates that can fit in a chip.

The manufacturing technology over the last decades has successfully reduced the area required to implement a transistor (and therefore the area occupied by a gate) at a pace of half the space approximately every two years.

This is the reason why the circuits are known as integrated circuits or chips. One of the consequences of such a large scale of integration is the possibility of designing circuits that contain hundreds of millions of transistors. Some of these circuits are the microprocessors powering the digital devices in use these days.



2 Impact facts derived from how the circuits are built:

1. Gates occupy an area in the silicon chip
2. Time to switch between the isolated and conducting stages

Factors that affect logic design:

- Circuits with a larger number of gates will react slower
- Smaller circuits will occupy less space in the chip
- Signals have to travel between gates, the longer connection, the larger the delay.
- Those with a large number of inputs have a larger switching time.
- Each gate consumes power while switching its value. The more gates in a circuit, the more power it consumes.
- Speed of transmutation of signals

Circuits need to be as compact as possible (least number of gates), gates close to each other, switch values as fast as possible.

Examples of combinational circuits

- Two bit **Adder** with Carry-in and Carry-out

In Boolean algebra the addition of two numbers x, y follows the same rules as conventional algebra. If we take a look at the last row of the table, if both numbers are one, the result of the sum is zero, but, if adding numbers with more bits, a carry value would be produced that should be added to the next digit.

Now imagine that you are performing the addition of numbers of various binary digits. For a pair of bits in the middle of the operation, we may have a carry bit produced by the previous bit addition, and propagate a carry bit for the next bit addition.

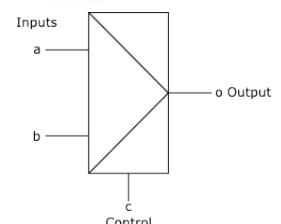
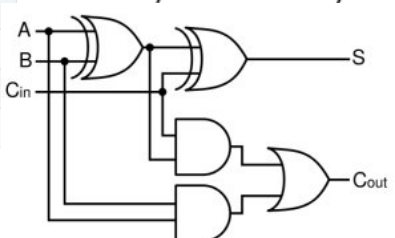
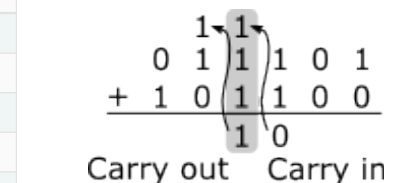
The previous truth table can then be extended to include an additional input which is the carry produced by the previous addition.

The translation of these two truth tables (one for each output) to digital gates can be done using the canonical representations (sum of products or product of sums) or with arbitrary gates.

- **multiplexor**: When the control signal is one, the output takes the value of the input a , otherwise, it takes the value of input b . —>
- **shift register**: A combinational circuit is needed that receives two bits as inputs and shifts them in the two possible directions depending on a third input. The new bit to be added are always zero.

A	B	S	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Inputs			Output	
A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



ELEC 1601 Week 5

Sequential Digital Circuits

Combinational circuits: once a combination of values is set at the inputs, the output is either zero or one depending on its truth table. The time it takes for the circuit to react to a change in the inputs is almost negligible (although designers care much about reducing it!).

A sequential digital circuit is a system that under certain conditions, the value of its outputs change depending on the inputs, or they *remain unchanged* (they are remembered, different from digital circuits which continuously reads the value) over time.

Navigation:

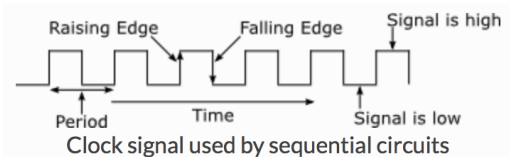
- The clock
- Flip-Flop
- FSM

The clock

The **sequential digital circuit** system works in two modes:

1. Ignoring changes in the input values and remembering the last result.
2. Or Paying attention to these inputs and adjust the output accordingly.

The signal that forces a sequential circuit to behave in one of the two modes is generically known as the **clock**. Every sequential circuit has a signal called the **clock** that signals when to pay attention to the value of the inputs and change the output, and when to ignore the inputs and maintain the value of the output.



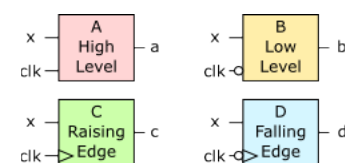
The most important properties of this signal are:

- Over time, the value of this signal oscillates from high to low level.
- The transitions from high to low levels and vice versa occur almost *instantaneously* and are called falling and raising edge respectively.
- The **period** of a clock (T) is the time it takes the signal to make a full transition (that is, from low to high, and back to low). The **frequency** of a clock (f), is the inverse of its period, that is $f=1/T$
- The frequency of a clock signal can be seen also as the **speed** at which it switches between high and low values. If a digital circuit is operating with a 5 GHz clock, it means the signal is making 5×10^9 full transitions (from low to high and back to low) per second.

How it works?

- At a certain *instant* marked by the clock, the circuit behaves as a regular combinational circuit and the outputs take the value from the inputs. We will call this instant **evaluation time**.
- At any other times, the value of the inputs are ignored, and the outputs remain with the values calculated in the previous instant. We will call this instant **memory time**.

We can assume that a sequential circuit is continuously changing from evaluation time to memory time at a pace defined by the clock signal.



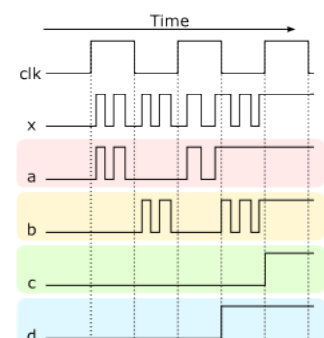
- Level-Sensitive Sequential Circuits

This type of circuits evaluate the inputs when the clock is at a high/low level. At any other time, the outputs are fixed to the values of the previous evaluation time.

- Edge-Sensitive Sequential Circuits

This type of circuits evaluate the inputs when the clock makes a transition.

If an inverter is connected to any of the circuits above, the circuit turns to the converse one.

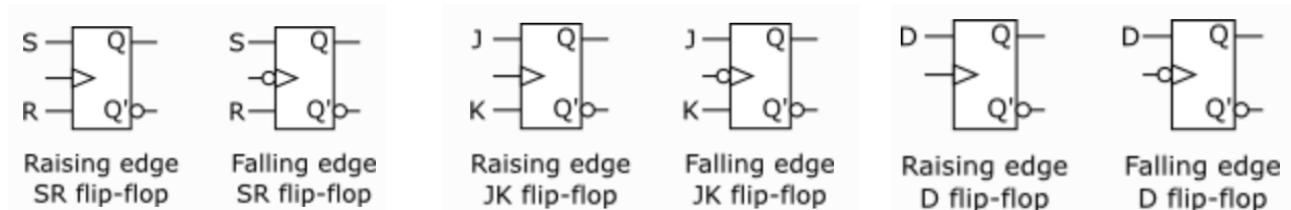


Flip Flops

A **flip-flop** is a sequential circuit that is *edge sensitive* and has one or two inputs (aside from the clock) and two outputs. The two outputs always provide one the opposite value of the other. The behaviour of a combinational circuit (or a Boolean expression) was captured by the truth table. The output column of the truth table contains the future value of the output. The convention that is used to represent this column is by denoting the output with the subindex $t+1$.

A flip-flop only makes a transition when allowed by an **edge** in the clock.

1. SR-Flip flop: 2 inputs(+the clock), S & R
2. JK-Flip flop: 2 inputs(+the clock), J & K
3. D-Flip flop: 1 input(+the clock), D



Examples of circuits with Flip-flops:

1. A Register

- The purpose of this circuit is simply to store a number of bits given by an equal number of inputs.
- Unless noted, register are typically implemented as several D flip-flops.
- The representation of a register storing 8 bits is drawn simply as a box with a clock signal and with a label in the arrow denoting the number of signals.

2. Stable Circuit

- The circuit will remain with the output *at the same value for ever*.
- We need to know the initial value of Q to know the future evolution of the circuit.
- Although this circuit is not very useful it shows one important property of edge sensitive sequential circuits, they can use their own output to compute the value at the inputs. It works because at the instant in which there is an edge in the clock, the new value is stored in the flip-flop, and even though the input may change, it is passed the edge and the change is not registered.

3. Switching circuit

- The output will switch at every rising edge of the clock.
- In this case, the input D of the flip-flop receives the negated value of the output.
- The initial value of Q should be defined.

4. Boolean function as input and Boolean function with state inputs.

- A flip-flop (in this example D flip-flop) may have as input a Boolean function itself with inputs coming from the outputs of the flip-flop.

The truth table of this circuit is derived by considering the output of the flip-flop as a regular input to compute the value of Q_{t+1} .

This table encodes the same behaviour as the one defined in the truth table of the JK flip-flop.

Truth Table of SR-Flip flop

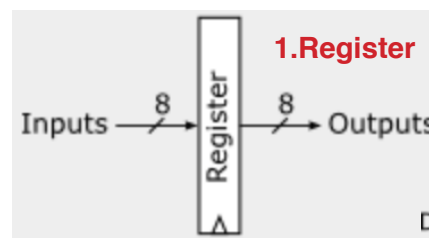
S	R	Q_{t+1}
0	0	Same Q
1	0	1
0	1	0
1	1	Undefined

Truth Table of JK-Flip flop

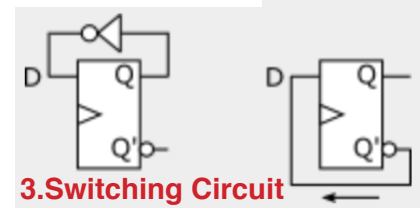
J	K	Q_{t+1}
0	0	Same Q
1	0	1
0	1	0
1	1	Opposite Q

D-Flip flop

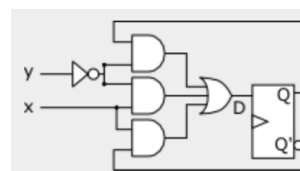
D	Q_{t+1}
0	0
1	1



2.Stable Circuit



3.Switching Circuit



Inputs			Output	
x	y	Q_t	Q_{t+1}	
0	0	0	0	
0	0	1	1	
0	1	0	0	
0	1	1	0	
1	0	0	1	
1	0	1	1	
1	1	0	1	
1	1	1	0	

Finite State Machines

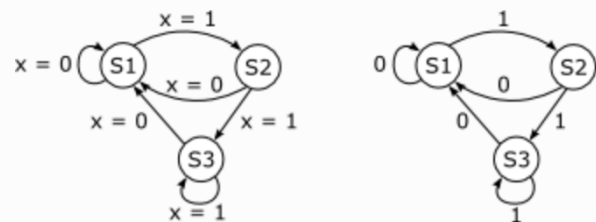
The truth table of a sequential circuit is a way to express the transition that will occur given the current value of the circuit, and the value of the inputs. This **transition** can also be seen as a change of **state**. The flip-flop is in a given state (defined by the value of its outputs) and after the clock edge, it transitions to a new state defined by its truth table. This notion of transition is extremely common when designing sequential circuits, to the point of existing a mathematical notation and a corresponding visual representation. They are known as **Finite State Machines**. The **label** in FSM edges represents the condition on the inputs that enables the transition. The **circles** represent the states and each of them corresponds with a different value of the **output** of the flip-flop.

The **arrows** represent the transitions in the flip-flop output values when a rising edge occurs.

In a generic FSM a state may have several arrows going out, and each one is selected based on a combination of the input variables. Consider the following FSM with three states and one input:

The representation on the left states explicitly the assignment for the input x to decide which transition should occur at each state. The representation in the right assumes that x is the input and only shows its value.

The interpretation of edges with these labels is very simple. If the FSM is in a state, the next state will be that obtained by following the arrow with the given input value. The transition will occur when the appropriate clock event occurs.



Not every edge in a FSM must have an expression stating when this transition is occurring.

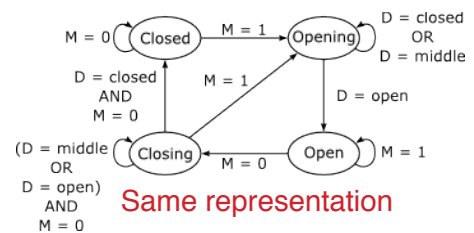
The number of transition out the states in a FSM depends on the number of inputs in the system.

Designing a FSM (Take automatic door for example)

Step	Description	Example
1	Identify the different states in the system. This is done by identifying when the system is carrying out some task, or waiting for some event to happen.	There are 4 states a. The door is closed. b. The door is in the process of opening c. The door is totally open. d. The door is in the process of closing.
2	Draw each of the states with a name in a circle.	We draw these states in circles with Closed, Opening, Open, and Closing.
3	Identify the system inputs. These are values that are obtained from the environment and changes in any of them prompt our system to change its current state.	The system has 3 inputs a. signal M from the motion detector. 0 for no motion, and 1 for detected. b. signal P from the door motor, 1 for completely open, 0 otherwise. c. signal C from the door motor, 1 for completely closed, 0 otherwise.
4	For each state consider how the system should evolve for every possible combination of the input values.	
5	Label each transition connecting two states with the input values that are required.	

We may assume that the frequency at which the FSM is going to evaluate its state and decide if it needs to change its state is higher than the frequency at which the events occur in the environment of the system being modelled.

The inputs do not necessarily need to be encoded as binary values, they can take any value from a pre-defined set. The translation of symbols, both as states or input values, to a binary representation is relevant when a FSM needs to be implemented as a sequential circuit.



From FSM to a Sequential Digital Circuit(Take right bottom FSM example)

Step	Description	Example
1	Identify the possible input values, and the number of states.	There is only 1 input with values A and B There are 3 states, S1, S2, S3
2	Encode the input values using binary logic.	We will encode as A=0, B=1. Denote by the name i.
3	Assign each state a binary combination (encode the states in binary).	We will encode S1 = 00, S2 = 01, and S3 = 10. Denote first bit S0, second bit S1
4	Create a truth table with as many rows as possible combinations of inputs and states. Aside from one column for each bit in the inputs and bit encoding the state, there should be a column for each state bit representing the next state .	See truth table at the bottom. i represents input value whether A or B. S0t S1t together represents States, for example, S0t=0, S1t=0 means state S0. The output column represents next state. The values represented by X in the output columns represent conditions that will not occur, so any value is valid.
5	Populate the truth table by assigning to the columns representing the next state the proper encoding representing each transition.	
6	For each column representing the next state translate the resulting values into a Boolean expression, and itself into a circuit.	Implementation of FSM with digital logic
7	Connect each of the resulting circuits to the input signal of a D flip-flop.	
8	Connect the output of the flip-flop to those inputs that represent the current state.	

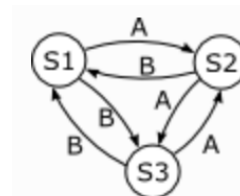
Output of FSMs:

The most generic application for a FSM is to receive inputs, based on those and the current state select a transition, but also produce some outputs.

Depending on the signals that are considered when producing the outputs there are two types of FSM:

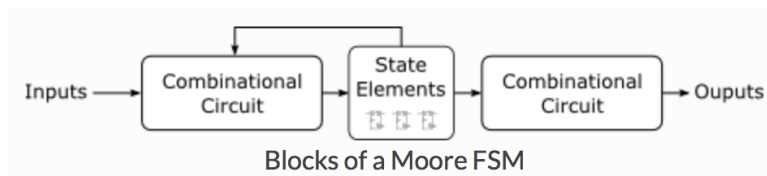
1. Moore machines

- Produce their outputs only using the signals that encode the state of the machine.
- The values of the outputs do not depend on the FSM inputs, they only depend on the value of the current state.



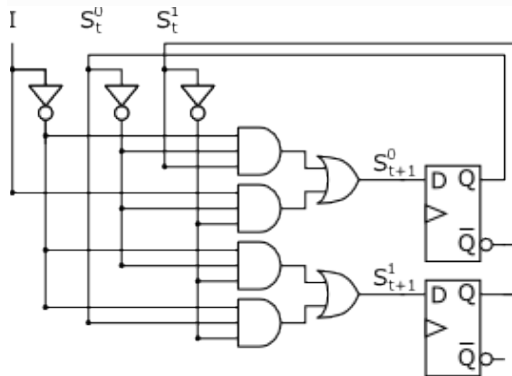
Example FSM

Inputs			Outputs	
I	S0t	S1t	S0t+1	S1t+1
0	0	0	0	1
0	0	1	1	0
0	1	0	0	1
0	1	1	X	X
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	X	X

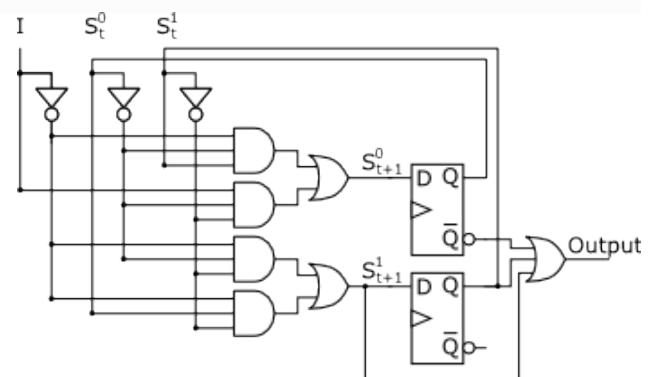
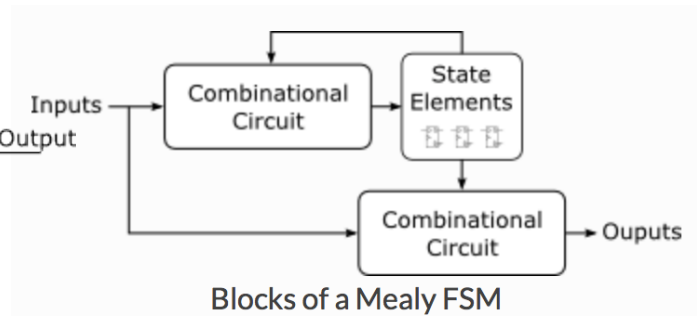


2.Mealy machines

- Produce their outputs that may depend on both the current state and the value of the inputs.
- In a Mealy machine, while being in a state, if the value of the inputs change, the value of its outputs may change as well.



change, the value of its outputs may change as well.



ELEC 1601 Week 7

AVR Architecture and Execution Environment

Navigation

- The Execution Environment
- The Execution Cycle
- The Stack

The term **architecture** is used to denote a generic description of a set of microcontrollers that are capable of executing the same machine instructions but that may differ in some characteristics such as the size of the memory, speed, etc. The architecture of a microprocessor consists of several modules that allow the execution of the instructions in its machine language. The **AVR architecture** refers to a family of microcontrollers that process most of their data in sizes of 8 bits, and it has a reduced set of instructions with fixed format.

When designing an advanced applications that make use of this chip. the most important source of information is the known as **Datasheet** containing the description of all the aspects of the chip. You may check the [Atmel 8-bit Microcontroller datasheet](#) to see the type of information included in these documents. Section 7 of that document contains a block diagram of the CPU, the description of the register file, the stack, and how are the instructions executed.

The Execution Environment

The **execution environment** of an AVR microcontroller refers to the *elements* used to execute instructions: type of data managed, data and status registers, available memory, etc.

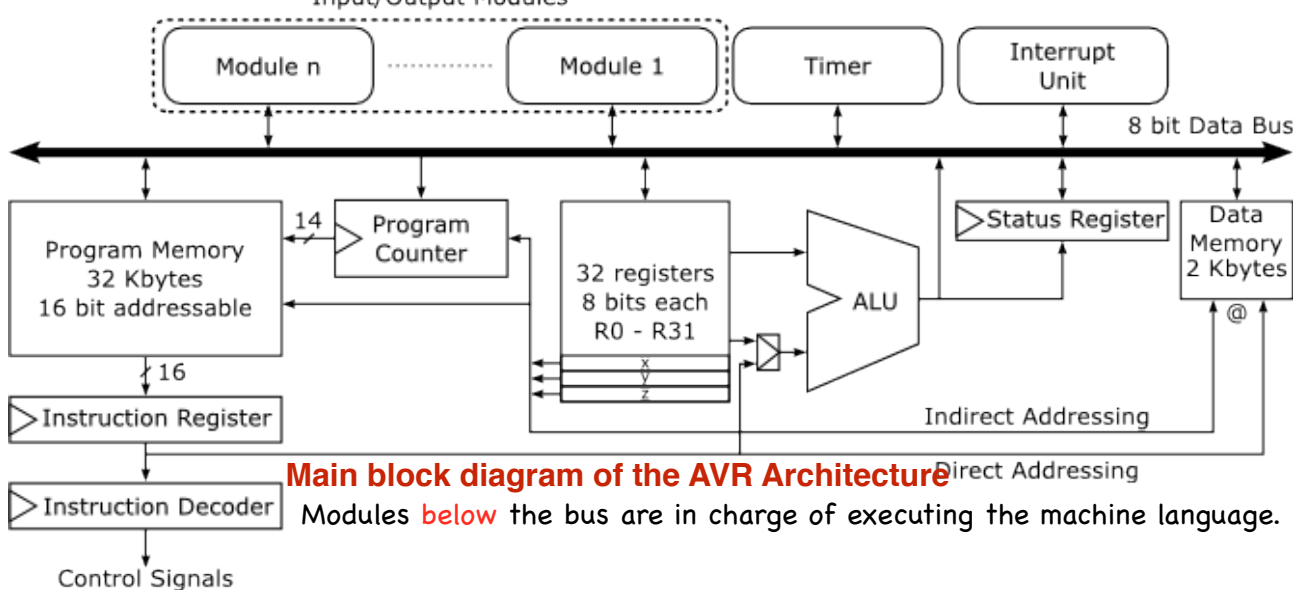
The units of information manipulated by a circuit are usually denoted by the terms in the right table—>

- The Data-path

In the AVR Architecture, the blocks are all interconnected through an 8-bit data bus through which data can be exchanged among all the **modules**.

Name	Bits
Byte	8 bits
Word	16 bits= 2 bytes
Doubleword	32 bits= 4 bytes
Quadword	64 bits = 8 bytes
Double Quadword	128 bits = 16 bytes

Modules **above** the bus execute other tasks such as input/output, timer events, interruptions....



Main block diagram of the AVR Architecture

Modules **below** the bus are in charge of executing the machine language.

- Program and Data Memory

Programs are stored in **Program memory** while data these program manipulate is stored in Data Memory. The machine instructions are obtained from the **Program Memory** and stored in the

Instruction Register from where they are decoded and the proper control signals sent to the rest of the components to execute the appropriate operation.

The memory stored in the AVR Architecture is **word addressable** (each cell with an address is of size 16 bits).

The size of both two memory varies depending on the model.

- The **Program memory** is a **flash** memory 16 bit(2 bytes) addressable. An address in this memory uniquely identifies a group of 16 bits.

Example: We will assume that AVR program memory has 32 Kbytes(2^{15} bytes), but since the cell is 2 byte, **addresses** for this memory require **14 bits**.

Thus, the range of values for the addresses is from 0x0000 to 0x3FFF. The read and write operations in this memory are then carried out with 16 bits of data. The code stored in the memory can be 16bits occupying one cell, 32 bits occupying two cells.....

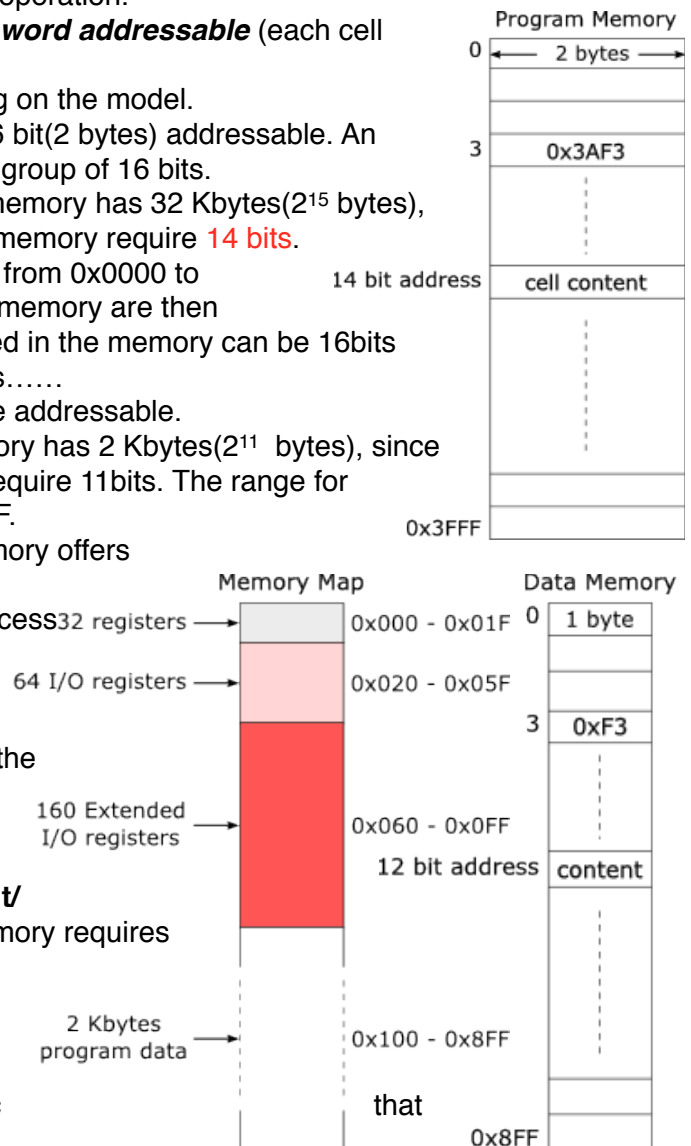
- The **Data memory** is a **static** RAM chip byte addressable.

Example: We will assume that AVR data memory has 2 Kbytes(2^{11} bytes), since the cell is 1 byte, addresses for this memory require 11bits. The range for address values *should be* from 0x000 to 0x7FF.

However, the chip is designed so that the memory offers addresses from 0x000 to 0x8FF. The first 256 positions(from 0x000 to 0x100) are used to access 32 registers **32 General Purpose Registers** and 64 additional input/output registers.

With this implementation, accessing to any of the 256 registers, or accessing data to its corresponding memory address in the data memory are exactly the same operation. This technique is known as **memory mapped input/output**. As a result, the addresses for this memory requires 12 bits.

- **EEPROM** (Electrically Erasable Programmable Read Only Memory) with a size between 256 bytes and 1 Kbyte and is used for basic does not need to be modified frequently.



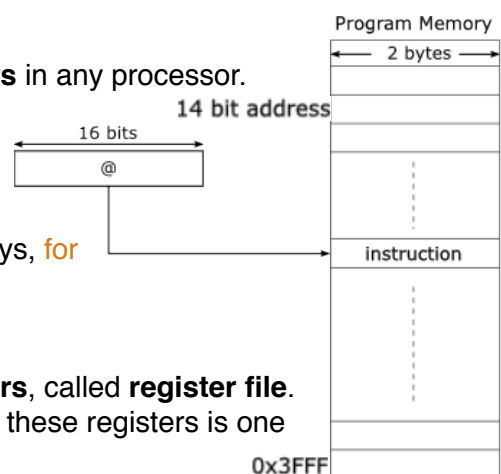
- **Program Counter**

The **program counter** is one of the most important **registers** in any processor.

It contains the **address** in the program memory of the next instruction to be executed.

The value of this register is modified by every instruction, as it has to point to the next one to execute.

Some special instructions modify this register in different ways, for example, the branching or jumping instructions(Week9).



- **Register file**

The AVR architecture has 32 **8-bit general purpose registers**, called **register file**.

The register are named ranging from R0 to R31. The size of these registers is one byte because the data memory is 1 byte for each cell.

The boxes are flip-flops. In the register file of the AVR architecture register R0 has address 0x000 because *it is mapped in the data memory*.

However, in some situations, the information stored in the register file is used to access the program memory and therefore 16 bits are required for the address or the data(Week10).

So R26 to R31 to be treated as if they were three consecutive 16-bit registers by concatenating two of them consecutively. Their names are X, Y and Z respectively.



Registers can also be accessed as if they were located in the lowest positions of the data memory, that is, addresses 0x000 to 0x01F. The figure points out the memory address of each register.

- Arithmetic/Logic Unit (ALU)

The Arithmetic/Logic Unit is **a combinational circuit** performing operations of three types:

1. Arithmetic. e.g. addition, subtraction with/without carry, and for signed/unsigned numbers.
2. Logical. e.g. bit-wise conjunction, disjunction, negation, 1's complement, 2's complement.
3. Bit functions. e.g. arithmetic and logical shifts, rotations, and byte swaps.

In the most advanced models of the architecture the ALU is also capable of multiplying two signed or unsigned integers.

The operands are obtained directly from the **register file**. e.g. `ADD R19, R20`(Week10)

For some instructions, one from the register file and the second from the **instruction register**(a field inside the encoding of a machine instruction).

- Status Register

It is convenient to reflect in some place that some *special situations* occurs during the execution of the programs. This place is usually the **status register**. These values are stored so that other instructions at a later stage can consult and perhaps make decisions based on those values.

For Example: The result of an addition has produced carry.

The way to offer this functionality is to refresh the content of the status register every time an operation is executed.

The status register of the AVR architecture contains the following eight **flags**:

C Z N: Indicates if the result of the last arithmetic or logic operation **C**occurs a carry, **Z**result is 0, **N**result is negative.

V: True for overflow when encoded in 2's complement

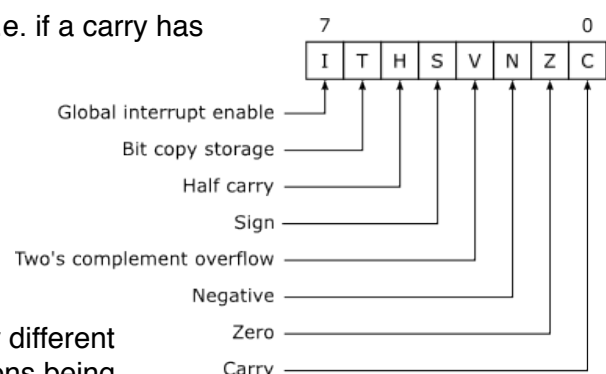
S: the sign of the result

H: if there is a carry out of the third bit of the result. i.e. if a carry has occurred at half the operator.

T: source or destination of bit copy operations `BST` and `BLD`.

I: if set, the interruptions in the microcontroller are processed. If zero, they are ignored.

The number of bits and conditions stored in this type of register is highly dependent on the architecture. Other architecture may have a radically different status register both in size and in the type of conditions being stored.

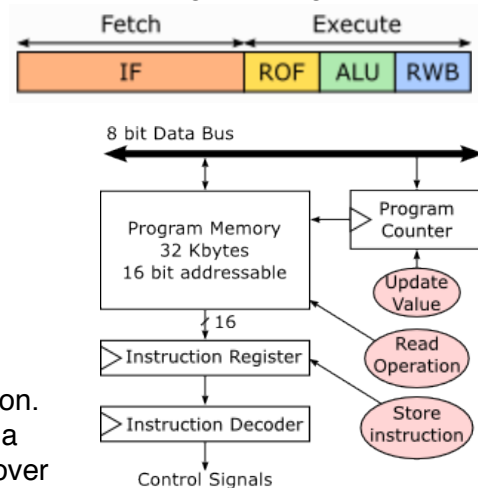


The Execution Cycle

The **execution cycle** of a microprocessor in general refers to *the internal steps to execute an instruction*. The number of these steps and their duration vary significantly from processor to processor and are totally dependent on the architecture. Most of the techniques used to increase the performance of a circuit focus on how to optimize the execution cycle.

The execution of most of the instruction in the AVR architecture is divided into two steps:

1. Instruction Fetch(IF): The instruction is obtained from the program memory and stored in the **instruction register**.
2. Execution: The instruction gets decoded and executed. For a typical instruction requiring an operation in the ALU, the second stage is further divided in the following sub-stages:
 1. Register Operand Fetch (ROF)
 2. ALU Execution (ALU)
 3. Register Write Back (RWB)



• Instruction Fetch(IF)

The processor obtains the next instruction(as well as the instruction type) to execute from the program memory.

1. The value stored in the **program counter** is given as address memory and a read operation is started.
2. The result of the operation is stored in the **instruction register**. At the same time, the **program counter** is automatically **updated** to point to the next memory location.

The controller contains a sequential digital circuit (similar to a FSM) that receives as inputs the instruction and generates over the next clock cycles the appropriate control signals for the rest of the components in the datapath to perform the required operations. This sequential circuit is one of the most important components of the architecture as it captures both the structure of the datapath (because it provides control signals to all their components) and the set of instructions (because it must be capable of interpreting all of them).

Some special operations have 32 bits in size(Week10) and therefore require an additional value from the program memory. In these cases a second fetch stage is executed, the value of the program counter updated, and the two bytes stored as well in the instruction register. This extra stage takes the same time as the first one.

The following 3 stages are valid only for instructions that require the use of the **ALU** and a result written back to the register file. Other instructions have different stages but with similar timing.

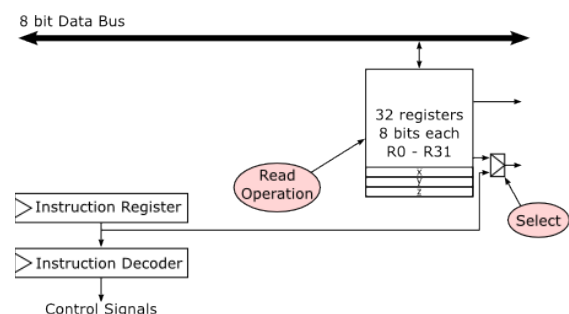
• Register Operand Fetch(ROF)

The **operands** that are required by the instruction are obtained from various sources. Most of the instructions use the values stored in the **register file**. This is the reason why this module has been designed with the following modes of operation with respect to inputs and outputs (to be processed by the ALU):

- One 8-bit output, one 8-bit result input
- Two 8-bit outputs, one 8 bit result input
- Two 8-bit outputs, one 16-bit result input
- One 16-bit output, one 16-bit result input

For example, if an operation requires to 8-bit values stored in two different registers, the register file can output both of them simultaneously into the ALU. For the same instruction, an 8-bit value can be written in another register.

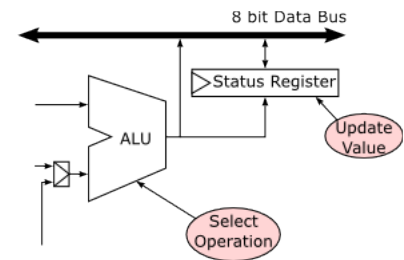
Some instructions contain one operand that is not in a register but is part of the instruction itself. A pre-defined subset of the instruction bits is extracted and used as operand. (Week10)The instructions that read or write data from the Data Memory are special cases about this stage. The operands required for their execution are used to calculate the address of the position in memory from where the data will be read, where the data will be written.



- ALU Operation Execute (ALU)

This stage is only present in those instructions that require an operation to be performed by the **ALU**. The control signals select the appropriate operation and the result is produced at the output. Remember, **ALU** is a combinational circuit.

The **status register** is **updated** to reflect the conditions of the result just obtained. More precisely, some of the 8 bits in status register changed, commonly CZNVSH.



- Result Write Back (RWB)

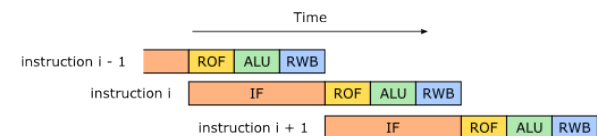
This stage is not present in all the instructions, only those that produced a result by the ALU in the previous stage that needs to be stored in the register file.

The ALU writes in the data bus the result. At the same time the register file selects the destination for the result, reads the value from the bus, and selects the write operation.

Pipelined execution:

AVR has a technique called 2-stage **pipeline** to improve the performance of the microcontroller. That is, to execute the two main stages of two instructions in parallel to increase speed. In other words, the execution stage of an instruction (containing ROF, ALU and RWB) is done in parallel with the fetch of the next instruction in sequence. This technique is similar to **product line**, and is now widely used by commercial microcontrollers and processors.

However, the execution of several instructions in parallel increases significantly the design of the overall circuit as the control signals need to be generated also in parallel and taking into account instructions that may break the normal sequence of instructions(jumps).



The stack

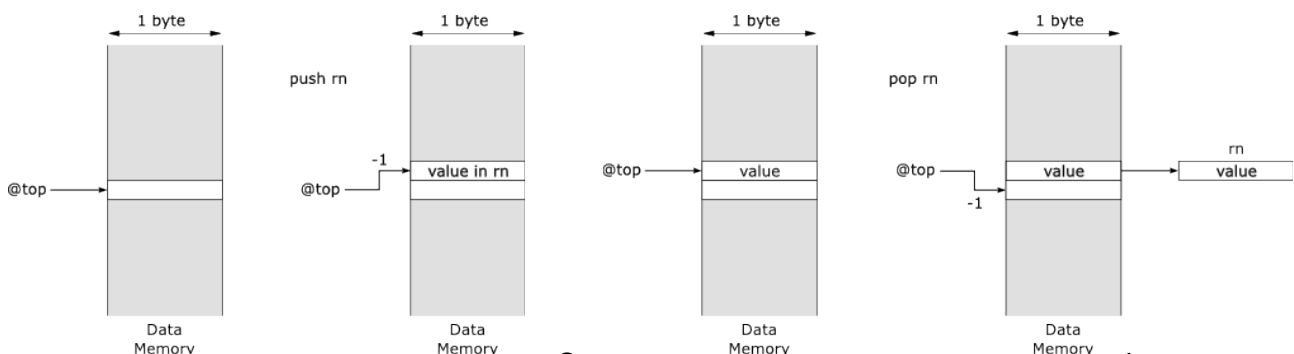
The **stack** is a portion of **data memory** to read and write temporary special instructions.

This area has a special location called *the top of the stack @top*.

- push R(Register), **write** something from the register R *on the top of the stack*. Decrease the stack point($@top = @top - 1$). The data in stack is lost. It doesn't change the value in register.
- pop R(Register), **read** from *the top of the stack* and write it in the register R. The effect is to store the content at the top of the stack in the register and adjust the address of the top. Increase the stack point($@top = @top + 1$). It won't delete the value in the data memory.

Push and pop instruction is restricted only for registers in AVR Architecture.

The data size is always 1 byte.

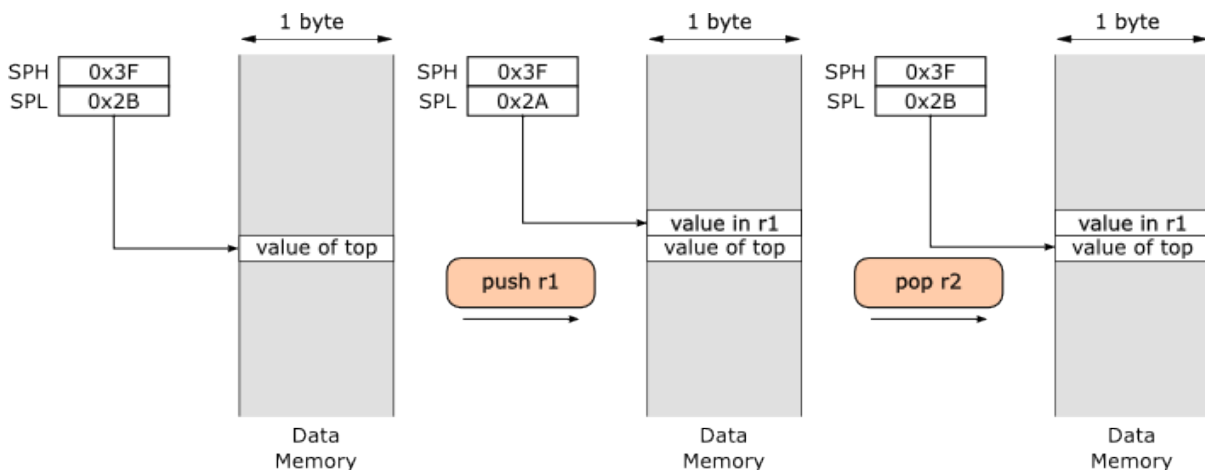


Some machine languages that allow operands of the stack instructions other than the general purpose registers.

The instruction push does not specify the destination operand, and the instruction pop does not specify the source operand. In both cases, it is assumed to be the element at the top of the stack. This means that the processor must keep the **address of this position in a register**. Such register is known as the **stack pointer**.

In the AVR architecture this register is part of the generic Input/Output registers and the value is stored in registers with addresses 0x3E and 0x3D. The reason to use two registers is because memory addresses are represented by 16 bits and we need two registers of eight bits each.

Register name	Address	Content
SPH	0x3E	Most significant 8 bits of the address of the top of the stack.
SPL	0x3D	Least significant 8 bits of the address of the top of the stack.



Stack and its address in AVR Architecture

Stack initialization, 2 steps:

1. Reserve memory specially for the stack.
2. Set the correct value in the stack pointer.

These tasks are typically performed at the level of either the operating system or in code not explicitly written by the user.

In the AVR architecture the stack pointer is typically initialized at the last position in data memory. It is assumed that the stack is empty and therefore the pointer will decrease its value as the stack *grows* in content.

The space for the stack is typically small. If an unusually large amount of data is pushed to the stack, its space may run outside of its bounds producing what is known as a **stack overflow**. See also INFO 1103(Week13)

ELEC 1601 Week 8

AVR Instruction Set Architecture

The set of instructions that a microprocessor can execute and their structure is generically called its instruction set architecture or **ISA**.

ISA contains all the details about the execution of each instruction and the effects in the components of the microprocessor. A microprocessor may have a small number of instruction types but a large number of total possible instructions because of the variations allowed in the operands (signed or unsigned, operand store in memory or general purpose registers).

For a complete document illustrating the instruction set, refer to [8-bit AVR-Instruction Set](#).

Navigation

- Types of Instruction Sets, CISC/RISC
- Instruction format of AVR Architecture
- The assembly language

Types of Instruction Sets, CISC/RISC

- Complex Instruction Set Computers (CISC)

The processors that provide a rich and complex set of instructions are called **Complex Instruction Set Computers (CISC)**. e.g. Intel's x86, IA-32, IA-64 (Itanium, x86_64 or AMD64). These processors are present in numerous desktop computers, servers, and laptops.

The machine instructions in a CISC computer allow handling multiple operands, some or all of them in memory, and accessed with address manipulations that may require certain arithmetic operations. For example, the following instruction is part of the IA-32 architecture:

ADD \$4, 14(%eax, %ebx, 8)

This line Adds value 4 to the data in memory (stored in the address obtained by adding the value 14, the content of register %eax, and content of register %ebx*8). The execution of this instruction requires more than one arithmetic operation. The ADD can be performed once the operands are obtained, and additional additions, a multiplication...

- Reduced Instruction Set Computers (RISC)

RISC architectures appeared as an alternative to existing CISC microprocessors. e.g. MIPS, ARM, SPARC, AVR Architecture.

Microprocessors have a very reduced set of instructions performing each a very simple operation. The idea is to translate this simplicity in the functionality into a faster execution, and simpler structure in the microprocessor.

The comparison between CISC/RISC is to execute the same high level task in two processors and measure the amount of time it takes to finish the task, instead of the number of instructions executed. A reduced execution time doesn't mean RISC is overall faster to execute the whole program than CISC.

A simple set of instructions means that complex operations will require sequences of instructions, some of them are very large. They both have the expression power.

Size of the code doesn't matter overall execution speed.

- Instruction Encoding
- **Fixed length format:** Every instruction has exactly the same size. A small number of operands are allowed in every instruction. The number of different instructions is also reduced as a larger number of instructions translates into a larger number of possible cases to encode.
- **Variable length format:** Allow instructions to be encoded with different number of bits, so an instruction may have an arbitrary number of operands as they will be encoded with additional bits.

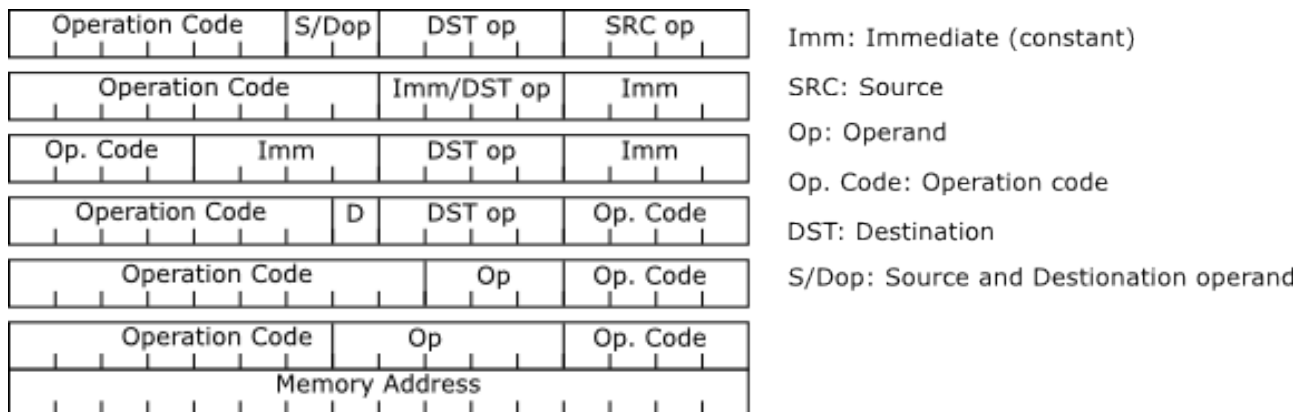
Characteristic	CISC	RISC
Time to Decode	longer	shorter
Succinct Code	less	more
Execution Speed per instruction	slow	fast
Overall Size	small	big
bits per Instruct..	bigger	smaller

The main **distinction** of two types occurs in the decoding stage. When the instruction has been loaded in the instruction register, the processor needs to identify which type of instruction it is and the type of operands that are required, then execute and obtain all its operands. A fixed format simplifies this stage as the bits encoding the different elements of the instruction are located in the same positions. While for Variable length format, the decoding stage may require loading additional bytes from memory while the instruction is being decoded.

Instruction format of AVR Architecture

The AVR Architecture has a **fixed length format**. Nearly all instructions(a few of them are 32 bits) are encoded with 16 bits. Here are different convention for registers and operands.

- Rd: A register in the register file for the **destination** of the result derived from the instruction.
- Rr: A register in the register file which will provide one of the **operands** for the instruction.
- R: Result of the instruction after its execution
- K: A constant value
- k: A constant memory address
- b: A bit in a register in the register file for an input/output register
- s: one of the bits of the status register.
- X, Y, Z: 16 bit registers obtained combining two registers in the register file.



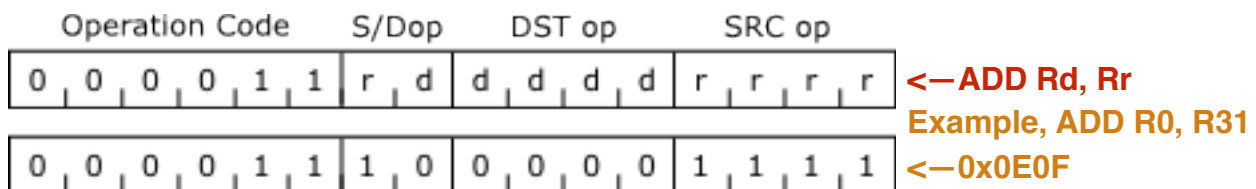
Components of Instruction in AVR Architecture:

Operation Code(op): Every instructions must have some bits to encode the type of operations(e.g. ADD, CPI).

Operands(SRC op, Op, DST). Instructions for which the operands are one of the 32 general purpose registers, require five bits per operand($2^5 = 32$). There are 3 types of operands:

- Sometimes these bits are not necessarily in contiguous positions. But this fact makes no difference when decoding the instructions.

e.g. **ADD R0, R31** $\rightarrow R0 = R0 + R31$.



The first operand is both a source operand and the destination in which the result will be stored. Letter r(source operand) has value 11111 referring to R31, letter d has value 00000 referring to register R0.

- Some instructions allow a number as operand. These numbers that appear literally in instructions are called **immediate** operands(Week10). These operands require the number to be encoded with bits.

e.g. CPI R16, 255—>Compare R16 with 255



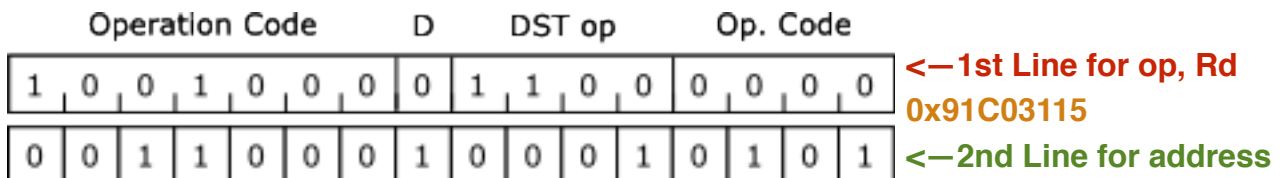
Internally, the microprocessor will carry out the operation R16-255 and update the status register to reflect the conditions of the result. Only certain conditions are captured in the status register.

Sometimes we may only have 4 bits for register in order to expand the range of immediate operand to ($0 \leq K \leq 255$). So instruction CPI can only use 16 of 32 general registers ($16 \leq d \leq 31$). This is an example of the special requirements derived from a fixed length instruction format.

So CPI R7, 255 will be incorrect as it doesn't have corresponding encoding in the AVR instruction format.

- This operands allow in some instructions are values that must be used as memory addresses. In this case, we need to extend instructions from 16 bits to 32 bits.

e.g. LDS R12, 12565—>Loads the content of memory in position 12565 into register R12



The operation code is represented by 11 bits, the leftmost 7 bits + rightmost 4. DST op. represents destination Register Rd. The second 16 bits encode the memory address.

After obtaining first 16 bits(0x91C0), the decoding stage detects a LDS instruction and therefore a second memory word needs to be loaded with the address operand. This double access to memory(fetch the instruction and to obtain the additional operand) will delay execution time.

The assembly language

Instead of simply writing 0 and 1 to represent instructions, we define a language with the same instructions, operands and formats as the machine language. It use combinations of letters and numbers so that it is understood by a programmer.

The translation between assembly language and a machine instruction is a straightforward application of simple encoding rules to transform each operand into a set of bits of the instruction format, as described above.

Each microprocessor has its own set of instructions and formats, thus, the assembly code is highly dependent on the architecture. However, there are several syntactical features that are common to almost all microprocessors. For example, instructions always start with the instruction mnemonic (ADD, CPI, LDS...) followed by the operands separated by commas.

The AVR assembly language has additional rules:

- The first operand after the instruction mnemonic is the **destination operand**. The remaining operands are **source operands**. There are maximum 2 operands in AVR Architecture.
- Registers are referred by their names between number 1 and 32 with prefix R or r.
- The assembly instructions are **not case sensitive**. R1, r1 or add, ADD are the same.
- Numeric constants in a instruction are simply represented as numbers with no prefix. CPI R16, 255
- Memory addresses are typically referred by its label which must be previously defined in the data section of an assembly program(Week9).

See Instructions of AVR Architecture for more details about some simple instructions.

ELEC 1601 Week 9

AVR Assembly Programs

Navigation

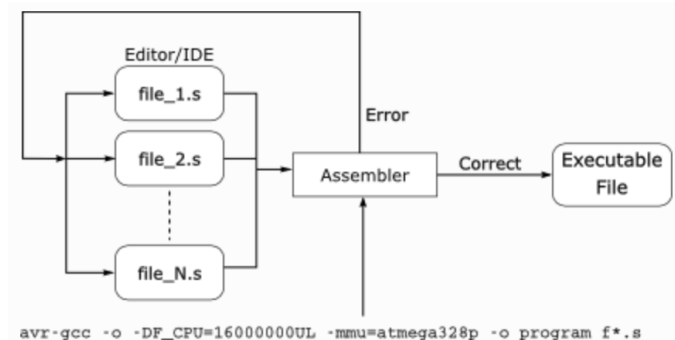
- Assembly Program Basic
- Create an Execution Program from Assembly code
- Data Definition
- Stack and Register Management

Assembly Program Basic

High-level programs need **compilers** to translate a program from one programming language to another, and produce another file with the translation(e.g. Java to bytecode, producing *.class file). Programs can be created using either a plain text editor(e.g. Notepad++), or an **Integrated Development Environment(IDE)**, a program with text editor+functionality to compile and upload i.e. Edit and process the files to generate an executable. e.g. Eclipse).

An **assembly program** is a readable representation of instructions in binary(e.g. ADD represents 0000 for operation code). It is quite close to the instructions executed by the processor. But it still needs to be processed in order to obtain a file that is suitable to *be stored in memory* and executed.

Assembly program is typically written in files with extension *.s(In comparison, Java program is written in files with extension *.java). **Assemblers** are such *programs* that given plain text file(typically *.s) containing assembly code, produce a binary file(*.exec) containing the binary encoding of all the instructions(i.e. machine language).



The syntax described below is that accepted by the **assembler avr-gcc**.

There are other assemblers that can also produce the translation but require the code to be written with slightly different syntax.

Create an Execution Program from Assembly Code

An assembly program is divided into sections. A program written in assembly code has a **data section** containing the data, and a **code section** containing the instructions to execute.

The words included in the program that are not translated into code but instead are order for the assembler are called **directives** (e.g. **.section**).

- **.section .data** does not translate into any instruction, but a mark to let the assembler know that the text after contains data definitions. All data definitions in an assembly program must be included in the data section.
- All the data defined in this section is stored in consecutive memory locations. If two variables are defined in contiguous lines, they will be stored next to each other in memory.
- This definition of a string is preceded by the label followed by string(s) and commas. This is the way a **label** is defined in an assembly code.

AVR Assembly Program Structure:

```
;; Data definitions go here
.section .data
```

```
;; Code definition goes here
.section .text
.global main
```

```
main:
```

```
ret
```

```
.end
```

This construction could be thought as the equivalent of declaring and defining a variable in a high level programming language(e.g. String m = "word";), but in a much more simplified way.

- **.section .text** tells the assembler that the data definitions have finished and the following lines contain the assembly instructions(i.e. code).
- **.global main** The following line contains one more directive declaring the word main as a *global* symbol. This means that the location in the code with such label **can be accessed from outside of the file**. (So that we can call main outside the file.)
- **main** is the mechanism used in assembly code to define the starting point of the program. By convention, the program will start in the instruction *labelled* with the name **main** and such label must be declared **global**.

The command that the assembler invokes the file.s:

```
avr-gcc -Wall -g -Os -DF_CPU=16000000UL -mmcu=atmega328p -o program file.s
```

- **avr-gcc** is the name of the assembler program.

The rest of the command are called **options**. They notify the assembler about specific requirements for the translation.

- **-DF_CPU=16000000UL**: The type of CPU that is being considered.
- **-mmcu=atmega328p**: The type of memory management unit.
- **program**: The name for the executable. And the executable with name **program** is created.
- **file.s**: The assembly files to compile.

If there is any syntactical error in the program, a message is shown in the screen and the program is not created. If the program is correctly written, the command finished with no message and the file has been generated.

Comments: `;;` at the beginning of the line or `;` The characters remaining in the line will be ignored.

Data Definition

The complex data structures(e.g. int, double...) offered by high level programming languages such as Java are translated in terms of simple memory definitions by the compiler. In Assemble Programs, however, *only simple data structures are allowed* as this language is so close to the representation used by the microprocessor. There are only 3 data structures(Integer, String, space)

• Integer Definition(.byte)

.byte defines numeric values of size *1 byte*. It can define one or several values separated by commas. **.byte** also allows different syntax(decimal, binary, hexadecimal, ASCII, octal...as below)

numbers: **.byte 38, 0b11011110, 0xFF, 'A', 0344, -12**

-12 is encoded in 8 bits in 2's complement. The previous directive is translated into the right table in memory. We can't control the address where these data are located. 0x120 is arbitrary.

- If a number larger than 255 is given, the assembler flags it as an error.
- If a negative integer is given that cannot be represented accurately in a byte, the number is truncated and a warning message is printed during the compilation process.
- If you want to store an array, we first should store the size of the array.

e.g. Let's assume we want to store int array[5] = {1, 2, 3, 4, 5};

.section .data

.byte 5, 1, 2, 3, 4, 5 ;the first 5 is the size

• String Definition(.ascii, .asciz, .string)

The definition of String is followed by a list of strings each delimited by double quotes and separated by commas.

- **.ascii**: Each character in the string is encoded with *1 byte* using the ASCII encoding using consecutive memory positions.
- **.asciz, .string**: Almost the same with **.ascii** but each string is encoded with **an extra byte** with value 0x00 as last character. It tells us where the string ends.

msg: **.ascii "S 1", "S 2"**

	1 byte
0x120	0x26
0x121	0xDE
0x122	0xFF
0x123	0x41
0x124	0xE4
0x125	0xF4
0x126	

```
;;S is 0x53, while space is 0x20.....
.asciz "S 3", "S 4"
;;Almost the same except additional 0x00 in the end.
.string "S final"
;;"S final" has 7 characters, as a whole occupies 8 bytes.
```

- **Empty space definition(.space)**

The .space directive followed by two numbers separated by a comma allows to reserve memory space with as many bytes as the first value and initialized with the second value. If the second number is omitted, the memory is initialized with default value 0.

```
result:.space 4, 0
```

```
;;Reserve 4 bytes space in memory with 0.
```

```
.space 4
```

```
;;Reserve 4 bytes space in memory with 0.
```

```
.space 8, 0xFF
```

```
;;Reserve 8 bytes space in memory filled 0xFF.
```

The main difficulty to manipulate data in assembly is that there is no information stored in *data memory* to remember the type of data or its structure.

An assembly program may access the bytes that encode the letters of a string and treat them as integers, naturals, floating point or any other available format. The data manipulated by any instruction is only referred by its address.

Memory

1 byte

msg:	0x53	0x20	0x31	0x53
	0x20	0x32	0x53	0x20
	0x33	0x00	0x53	0x20
	0x34	0x00	0x53	0x20
	0x66	0x69	0x6E	0x61
	0x6C	0x00		

Memory

1 byte

result:	0x00	0x00	0x00	0x00
	0x00	0x00	0x00	0x00
	0xFF	0xFF	0xFF	0xFF
	0xFF	0xFF	0xFF	0xFF

Labels

Labels are strings that are written at the very beginning of a line in the program and are followed by a colon: . However, label itself doesn't store in the memory.

- Labels for data(memory address in data memory).

```
data: .byte 38, 0b11011110, 0xFF, 'A', 0344, -12
```

The label *data* represents the address of the *first* of these values. Knowing the size of the data, we can get the position of the rest of the elements by simply adding its offset from the first position.

Labels can be included directly as operands in instructions to refer to the data they point to.

```
LDS R12, data ;;This would load with value 0x26 from data memory.
```

```
STS data, R12 ;;This stores the content of R12 in the memory location
data points to. We don't need to know the exact address.
```

- Labels for code addresses can be used as operands of subroutines or conditional branches. They mark certain location in the code and then refer to it to *jump destinations* or make a function call. e.g. BRSH data, CALL function.

1 byte

data	0x26
@data + 1	0xDE
@data + 2	0xFF
@data + 3	0x41
@data + 4	0xE4
@data + 5	0xF4
Address	

How to know the memory address that a label refers to?

If label is previously defined in data section:

hi8(label) returns the 8 most significant bits of the address assigned to the label.

lo8(label) returns the 8 least significant bits of the address assigned to the label.

In the AVR architecture, memory addresses have 16 bits and the general purpose registers have 8 bits, so a complete memory address has to be stored in two 8-bit registers X, Y, Z.

Example: Let's assume the address *data* refers to is 0xCCDD in data memory.

```
LDI R29, hi8(data) ;; R29 will be 0xCC.
```

```
LDI R28, lo8(data) ;; R28 will be 0xDD.
```

```
LDS R12, data ;; R12 will be 0x26—the first number 38.
```

```
.section .data
```

```
data: .byte 38,
0b11011110, 0xFF, 'A',
0344, -12
```

```
.section .text
.global main
```

```
main:LDI R28, lo8(data)
LDI R29, hi8(data)
```

When the program is loaded in memory and is about to execute, the address of the label in that instruction is then replaced with the actual location in which the data is stored. This technique allows users to write assembly programs knowing only the **relative position** of data with respect to labels.

LDD, STD allows to specify a constant(q , $0 < q \leq 63$) to be added to the register Y or Z.

e.g. LDD R20, Y+q. STD R21, Z+q

Let's assume the address of the label has been pre-loaded in these two registers using LDI R29 hi8(data), LDI R28 lo8(data).

LD R11, Y	ST Y, R11	LD R11, Y
LD R12, Y+	ST Y+, R12	;;R11 will be 0x26,
LD R13, Y+	ST Y+, R13	;;which is equal to 38
LD R14, Y+	ST Y+, R14	LDD R16, Y + 5
;;Load the values from	;;Store the	;;R16 will be 0xF4,
data memory to Register	values back to	;;which is equal to -12
and increment the	memory and	ST Y, R16
address by 1 every	increment the	;;Store 0xF4 in Y
time.	address by 1	STD Y + 5, R11
R11 will be 0x26, R12	every time.	;;Store 0x26 in Y + 5
0xDE and so on.....		;;Swap 2 elements in memory

In left case, register Y is increased by each instruction, whereas in the right sequences, the register is left untouched.

Why q should be > 0 and ≤ 63 ? Because the label usually points to the top of an object. There is a restriction for q to encode. q can't be exceeded to 6 bits according to Instruction set(Week10).

Labels can be considered as the basic mechanism used by compilers to declare variables. Each variable in a program has a memory location which could be assimilated to the address of the label. However, high level programming languages place additional information in a variable such as its type that is used for multiple checks that are not possible in assembly programs.

Labels must be *unique* throughout the entire file containing the assembly code and can be used indistinctly in the data or code part.

The type of operations allowed in the parameters is different for each microprocessor and is one of the defining features of the architecture.

Stack and Register Management

- Stack Restrictions

Handling the stack is one of these restrictions that is present at assembly level but is totally hidden when programming a digital system in a high level programming such as Java. The processor uses the stack to store additional data, which means not only pop/push instruction will modify the stack.

The top of the stack (the data pointed by the stack pointer) must be exactly the same before the first instruction of a routine is executed, and before the execution of the last instruction(RET).

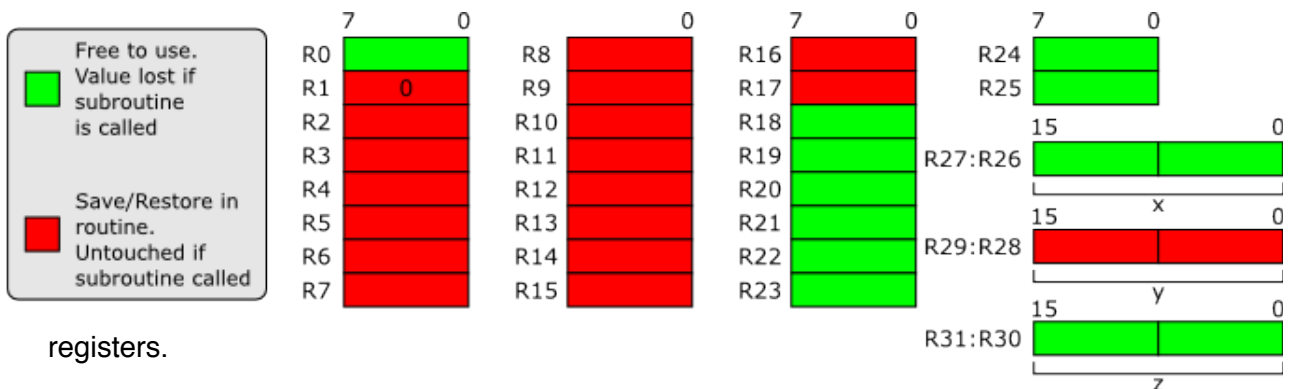
The main consequence of this restriction is that a subroutine will place on the stack some values that are used during the execution, and then remove all of them from the stack before the end of the subroutine.

The operating system running in a microprocessor is the entity in charge of preparing the executing environment for a program, and the stack is part of such environment. A portion of memory is reserved for the stack and the stack pointer is initialized with its address. Once the execution environment is ready, the routine with name main is then invoked, marking the start of the execution.

- Register Restrictions

This comes when we call a function or a subroutine where a subroutine may modify Registers. If R2 is going to be modified by the subroutine, we don't need to save R2.

1. Register R0 is used as scratch register and need not to be saved nor restored.
2. Register R1 must be kept always at value zero.
3. Registers R18 to R27, R30 and R31 may not be saved before its use, but they may be overwritten when a function is called from your code. It depends on you that to reserve or not.
4. Registers R2 to R17, R28 and R29 *must* be saved and restored at the beginning and end of a subroutine.
5. If a function returns an integer as result, this must be placed in register R25:R24 at the end of the subroutine. The calling subroutine will expect that result to be present in those



A few tips for designing an Assembly Program

```
;;copy the content of registers in the stack
push R24
push R25
ldi R24, lo8(my_msg)
ldi R25, hi8(my_msg)
push R25
push R24
call printf
;Call a subroutine

;After call
;Remove the values in the stack point
pop R0
pop R0
;;restore the content from the stack
pop R25
pop R24
ret
```

- Make sure we leave the stack pointer at the end of the routine in exactly the same location as it was at the beginning of the routine. So that the subroutine returns to the correct location.
- Before execution/subroutine, we should copy the content of registers(R2 to R17, Y) in the stack.
- Avoid unnecessary operations. *For example*, do not save and restore registers that are not needed or the content of which is not important. Try to move the data to the right location to begin with to avoid unnecessary data movement operations.
- There is always more than one way to program a task. If possible, choose the one with less number of instructions or that you think it will execute faster. Memory accesses typically slow down program execution.
- Write legible code. Insert comments before blocks of instructions so that you know what is the purpose of several instructions. Do not include trivial comments about a single instruction, but instead, high level comments about code blocks.

ELEC 1601 Week 10

Addressing Modes of AVR Architecture

After an instruction is loaded from memory in the instruction register, the microprocessor first decides which instruction is, and then obtains the required operands. This section will discuss this problem: How to obtain operands? How to efficiently obtain operands using simple manipulations? Calculations in instructions to obtain operands are called **Addressing Modes**.

The complexity of the addressing modes is usually limited by the size of the microprocessor. Implementing these modes requires additional circuits to perform auxiliary arithmetic operations and therefore designers must strike a balance between the complexity of the modes, and the complexity of the resulting circuit.

The variety of methods and limitations to access these operands change from one microprocessor to another. Operands are typically stored in 1) general purpose registers, 2) in memory 2) in the instruction itself. Accessing the general purpose register or the instruction itself is very simple. However, when the operand is in memory there are certain additional operations to manipulate the address that can be done very efficiently.

Navigation

- Notations to describe Addressing Modes
- Addressing Modes in the AVR Architecture

Notations to describe Addressing Modes

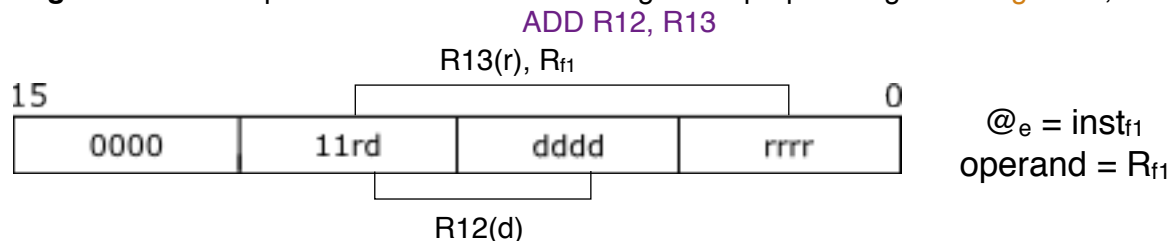
- Effective address of an operand (where the operand is stored): $@_e$. This address is not restricted to memory address. The simplest addressing mode will obtain operands from registers, so $@_e$ may point to a general purpose register.
- Instruction used to calculate $@_e$: $inst$, and assume the instruction is stored in the address $@_{inst}$. This instruction will have certain fields, or set of bits, used to calculate $@_e$. Those fields will be denoted to $inst_{f1}$ to $inst_{fk}$.
- If field fi encodes a general purpose register, that register will be denoted to R_{fi} .
- $DMEM[a]$ & $PMEM[a]$ refers to the content stored in the data & program memory in position a .
- Loading the value v in register R will be denoted by the expression $R \leftarrow v$

Addressing Modes in the AVR Architecture

The operands present in one machine instruction of the AVR architecture can be divided into four categories: registers, constants, memory addresses, and implicit operands.

Except for the implicit operands (e.g. RET), the rest are obtained using Addressing Modes.

- **Register Direct**—operands stored in one of the general purpose registers. e.g. ADD, SUB



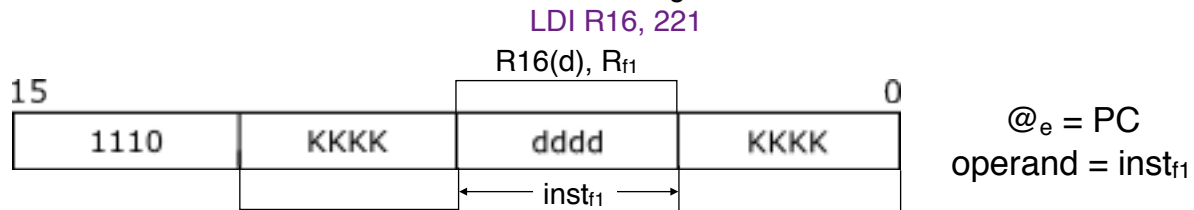
In this case $@_e$ is a register address inside the register file.

- **Immediate**—constant is already in the instruction. e.g. LDI.

This addressing mode simply provides the operand directly extracting a set of bits from the instruction. This is a case that the data used in a program is extracted from the program memory. The effective address of the operand $@_e$ can be considered the value of the program counter (the address of the instruction itself) as it contains the position in the program memory in which the instruction being executed is stored.

Similarly, the operand is simply the value of the instruction field (The Constant). The value of the operand is going to be limited by the number of bits allowed to encode the value.

In the AVR architecture the immediate mode can only be used when loading a value into a register with the instruction LDI. The constant is encoded using 8 bits from 0x00 to 0xFF.



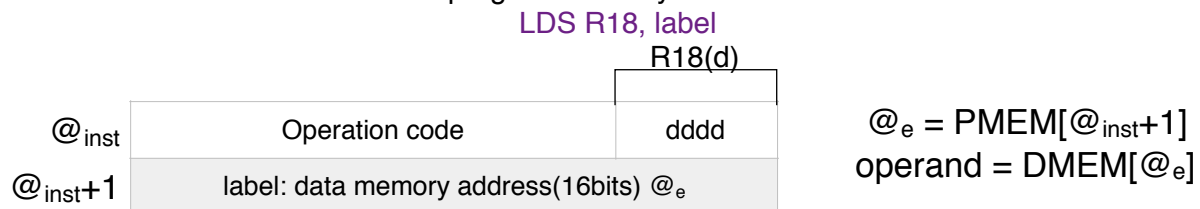
Due to the size of the other fields in the instruction (operation and immediate operand), there are only 4 bits available to specify the register to store the value given in the instruction. As a consequence, this instruction is restricted to use only registers R16 to R31 as destination register.

- **Data Direct**—memory address in data memory as part of the instruction. e.g. LDS, STS

The effective address of the operand is obtained from the instruction itself.

The AVR architecture uses 16 bit addresses, therefore, the size of a instruction is **32 bits**.

The instruction contains the address of the data memory in the third and fourth bytes, but since the program memory has cell size of 2 bytes, they address is stored in the next position. This is the reason why there is one unit added to obtain the effective address. The operand is in data memory, but we use an address stored in the program memory to access the data.



R18 is an operand using Register Direct with only four bits(R16 to R31).

This addressing mode requires an execution cycle longer than other instructions because it requires 3 memory access. Aside from the 2 accesses to the program memory to get the instruction(@inst, @inst + 1), a third access to the data memory is required to either read(LDS) or write(STS) the operand.

Additionally, the execution of this instruction requires the program counter to be updated twice. The first time is to access the address stored in the program memory, and the second to leave it pointing to the next instruction.

- **Data Indirect**—memory address of data memory in Register X, Y, Z. e.g. LD, ST

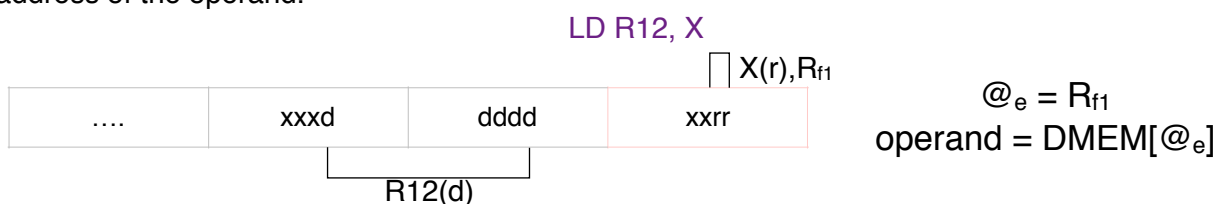
Access an operand stored in the data memory but uses as direction a value stored in one of the three 16 bit registers contained in the register file, X, Y, or Z.

The effective address is directly the content of the register X, Y, Z encoded in the instruction field.

The operand is then obtained from the data memory using the content of the register as address.

The register used in this addressing mode is supposed to have a valid address *that has been previously loaded*. For example: LDI R26, lo8(number), LDI R27, hi8(number).

The field inst_{f1}(two bits, 00 for X, 01 for Y, 10 for Z) encodes the register to use to obtain the address of the operand.



The instructions using this addressing mode are more compact than those using the data direct mode because instead of encoding an entire memory address, they only need to encode one of the 3 16-bit register. In the example this mode is encoded in 2 least significant bits.

Additionally, the instruction uses register direct to obtain the destination operand.

This addressing mode requires one additional memory access. Upon decoding the instruction the microprocessor sends the value of the register as address and performs the requested operation. This addressing mode is useful when accessing a memory address several times, or when the address of a data is the result of a previous arithmetic operation. In the AVR architecture, only X, Y, Z register can be used with this mode. In other architectures this restriction is not present and any general purpose register can be use for indirect use.

- **Data Indirect with Post-increment**—memory address of data memory in Register X, Y, Z Quite like the previous one, but additionally, the register containing the address is automatically incremented by one unit and updated with the new value after execution.

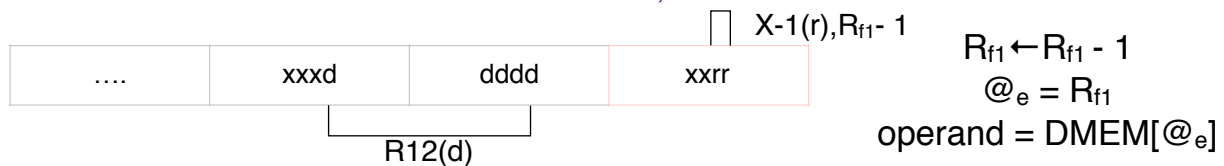
LD R12, X+



This addressing mode is very useful to access data that is stored in consecutive data memory locations and the address of its first element is already pre-loaded in one of the 16 bit registers.

- **Data Indirect with Pre-Decrement**—memory address of data memory in Register X, Y, Z Quite like the previous one, but decrease the address before the value of the register is used to access memory. The microprocessor performs this subtraction as part of the regular execution and before accessing memory.

LD R12, -X



This addressing mode can also be used to access elements in a sequence with the difference that the elements are accessed in decreasing memory locations.

Post-increment/Pre-decrement Summary:

Post-increment/Pre-decrement are executed as part of the instruction to load the data from memory. This increases the overall performance of the sequence.

As part of the execution cycle, the instruction requires to memory accesses, the one to obtain the instruction (instruction fetch) and the one to access the data memory operand.

POP and PUSH Addressing Mode		
Step	POP Rd(Post-increment)	PUSH Rr(Pre-Decrement)
1	Read data from top of the stack.	Decrement Stack Pointer(SP).
2	Store it in the register Rd.	Read data from register Rr.
3	Increment Stack Pointer(SP)	Store it in the top of the stack.

Side effect of using the post-increment/pre-decrement of the data indirect mode is that the original value of the address is lost. If the original value is needed, either you have to save a duplicate in another register, or pre-load its value again.

- **Data Indirect with Displacement**—memory address in Register Y, Z, e.g. LDD, STD The effective address is obtained adding two instruction fields, one that specifies a register, and the other encoding constant value. The result of the addition is sent to the data memory as address for the appropriate operation.

LDD R12, Y+37



This mode has an additional restriction: Only register Y and Z can be used for the indirection. The addition involves the 16 bit address stored in one of these registers with a 6 bit constant(q) extracted from the instruction. The size of this field limits the values allowed for the displacement q, from 0 to 63.

Instructions using this mode perform two memory accesses. The first one is the instruction fetch. The second is to access the operand once the addition of the address and the offset is computed.

This addressing mode is very powerful to access the fields of a complex data structure. When declaring variables, we know the address of first variable, and its variable type(int, double..., i.e. how many bytes this variable occupies), we can use this to access the second variable, third variable...

It keeps the address of the data structure fixed in one register and access the fields directly with the different displacements. No modification is performed in Y, Z.

*The reason why we can only use Register Y or Z is probably because we only need one bit to encode 2 registers so that we could have one more bit to do other things.

*Other Addressing Modes in the AVR Architecture

I/O Direct Mode: Used to access operands in the input/output register space. It is similar to the register direct mode.

Program Memory Constant Addressing: An address in register Z is used to access the program memory (instead of the data memory). This mode is reserved for a small subset of machine instructions.

Program Memory with Post-increment: Similar to the previous one. Only register Z can be used, and its value is incremented after the access.

Indirect Program Addressing: The content of the Z register is loaded in the program counter to access the program memory. This addressing mode is restricted to the machine instructions **IJMP** and **ICALL**.

Direct Program Accessing: The instruction contains a 16 bit number that is pointing to the program memory. The value is loaded into the program counter. This mode is restricted to the machine instructions **JMP** and **CALL**.

Relative Program Addressing: The instruction contains a constant that is added to the program counter, the result incremented by one, used to access the program memory, and the result loaded in the program counter. This mode is restricted to the machine instructions **RJMP** and **RCALL**.

ELEC 1601 Week 11

Translation of High Level Programming Constructs

The applications that you usually see executing in digital devices are generally all written in some high level programming language. These programs are then translated to the machine instructions by compilers or interpreters. These tools read multiple files written in one language and produce one or several output files in a different format, which may directly be executed by the processor. Languages such as Java are object oriented, that is, they allow the definition of classes, the grouping within those classes of fields and functions with different degrees of visibility, the creation of objects of these classes, etc. This type system requires programs to be written following a clearly outlined set of rules that are then checked by the compiler, and if they are not correct, the final program is not generated. Other properties such as variable names and visibility are also checked by the compiler during the translation process.

The crucial instructions to translate this construction to assembly code are the **conditional branch instructions**. They allow changing the sequence of instructions executed depending on the value of a flag previously set by a comparison operation.

Navigation

- If-else
- Switch
- While loop
- For loop

Assumption: The compiler has verified all these properties.

Variables has been previously defined in data section with label name(e.g. x: .byte 1, y: .byte 0)

Integers are 8 bits. True is 1, false is 0. Results after evaluation have been stored in R18.

If-then-else

If-else statement in Java

if (BOOLEAN EXPRESSION true or false) { // then Block } else { // else Block }	if((x <= 3) && (i == (j + 1))){ // Block A } else { // Block B }
---	---

Restriction: The expression in the first line must return a **Boolean value** after evaluation.

Semantic: Condition true, execute then Block, avoid else Block; Condition false, execute else Block, avoid then Block.

Assembly for if-else(4 versions for changing CPI and BREQ):

EXPRESSION EVALUATION	
ST R18, ... ;;Store result in R18	LDS R18, x
¹ CPI R18, 0 ;;0 for false	LDS R19, 3
² BREQ else ;;If false, go to else	CP R19, R18 ;;Compare if 3<x
;;We can Make multiple comparisons—>	BRLT else ;;If true(x>3), go to else
;;As well as some arithmetic operations—>	LDS R18, j ;;Keep comparing if first is true
	INC R18 ;;j = j+1
	LDS R19, i
	CP R18, R19
	BRNE else
then: then Block	Block A
.....	
JMP done ;;Skip else Block	JMP done
else: else Block	else: Block B
.....	
done: 	done:

¹CPI R18, 0 can be changed to CPI R18, 1

²BREQ else can be changed to BRNE.

The evaluation of Boolean expressions may, in some cases, translate into strange or non-intuitive behaviors. For example, consider the following condition in an if-then-else block:

<pre>if ((x < 3) && (j < k++)) { // Body }</pre>	<pre>if ((j < k++) && (x < 3)) { // Body }</pre>
--	--

If the compiler translates the evaluation of the expression as shown in the left example, the second part of the conjunction will only be evaluated if the first part is true. Therefore, variable k will only be increased if $x \geq 3$.

Although logically equivalent to the left one, the right sub-expression is evaluated always, and therefore, variable k always increases its value.

This property is known as ShortCircuitEvaluation. —Wikipedia [ShortCircuit.java](#)

Switch

In the switch statement, there are several blocks each of them including a value to use to compare with an expression given in advance.

Switch Statement in Java

<pre>switch (EXPRESSION) { case VALUE_1: Case_value_1 BLOCK break; // Optional case VALUE_2: Case_value_2 BLOCK break; // Optional case ... default: // Default block is optional Default BLOCK }</pre>	<pre>switch (x + y){ case 10: //Case10 Block Break; case 20: //Case20 Block Break; case 30: //Case30 Block, no break default: //Default Block }</pre>
--	---

Restriction: The EXPRESSION should return a value. The data type of the expression and those of the values in the cases have to be identical.

Semantics: Expression is compared **in order** with the value in case. If equal, jump to the corresponding block and keep executing until reaching a break(break can be optional). If no match, jump to default if there is a default Block. If there is a break in the block, jump to the end.

Assembly for Switch(BREQ is better than BRNE):

<pre>EXPRESSION EVALUATION ST R18, result CPI R18, value_1 ;;Compare first case BREQ case1 CPI R18, value_2 ;;Compare second case BREQ case2 ;;Compare other cases JMP default ;;If there is a default Block (JMP done) ;;If there is no default Block case1: Case 1 BLOCK JMP done ;;If there is a break case2: Case 2 BLOCK JMP done ;;If there is a break case... default:Default BLOCK ;;Optional done:</pre>	<pre>LDS R18, x LDS R19, y ADD R18, R19 CPI R18, 10 BREQ case10 CPI R18, 20 BREQ case20 CPI R18, 30 BREQ case30 JMP default case10:Case10 Block JMP done case20:Case20 Block JMP done case30:Case30 Block (No break no JMP) default:Default Block done:</pre>
---	--

We can change the orders of the case only if all blocks will jump to the end after execution(all cases have a break).

With this structure, the comparisons are included at the top of the translation and performed in the same order as written in the statement. The code blocks for the options are translated in consecutive locations. This layout provides the correct execution when no break statements are present. The presence of the break statement translates directly into an unconditional jump instruction to the end of the statement. If there is no default option, the labels default and done point to the same location, which is the desired effect.

While loop

While loop in Java

```
while (BOOLEAN EXPRESSION true or false) {    while((i < size) && (y < x)){
    // Loop body                                //Loop body
}                                              i++;
}
```

Restriction: The expression in the first line must return a Boolean value after evaluation.

Semantic: First evaluate conditions, if true, execute loop body, and evaluate again. If false, jump to the end.

Assembly for while loop(body label is useless)

<pre>eval: EXPRESSION EVALUATION ST R18, ... ;;Result is stored in R18 CPI R18, 0 ;;Or, CPI R18, 1 BREQ done ;;Or, BRNE done (body:) Loop Body JMP eval ;;Evaluate again done: </pre>	<pre>(eval:) LDS R18, i *eval: LDS R19, size CP R18, R19 ;;Compare i and size BRGE done LDS R20, y LDS R21, x CP R20, R21 ;;Compare y and x BRGE done (body:) INC R18 STS i, R18 JMP eval done: </pre>
--	--

Assembly for while, do-while loop(check the condition at the end of the block, eval is useless)

<pre>(JMP eval) ;;If it is a while loop, write this. body: Loop Body (eval:) EXPRESSION EVALUATION ST R18, ... ;;Result is stored in R18 CPI R18, 0 ;;Or, CPI R18, 1 BRNE body ;;Or, BREQ body done: </pre>	<pre>*LDS R18, i body: INC R18 STS i, R18 (eval:) (LDS R18, i) LDS R19, size CP R18, R19 BRGE body LDS R20, y LDS R21, x CP R20, R21 ;;Compare y and x BRGE body done: </pre>
---	--

Both 2 versions can all be used while the second version also applies to do-while loop. If we put JMP eval before body, it becomes a while loop.

Depending on the high level programming language, the evaluation of the Boolean condition in the first line of this statement will also suffer the Short-circuit evaluation effect.

*An optimisation of the previous assembly program is to put LDS R18, i in front of eval(version 1)or body(version 2). This can prevent R18 load from data memory over and over again. Because we only need R18 once when we start executing the loop. * means optimisation version, disregard().

For loop

For loop in Java

```
for (Initialization ; Condition ; Update) {  
    // Loop Body  
}  
for(i = 0; (i < size) && (j != 0); i++){  
    // Loop Body  
}
```

The statement consists of four blocks. Three of them are written in parenthesis in the first line and separated with semicolon. The fourth is simply the code block inside the statement.

Semantic: The first statement in parenthesis is executed only once and at the beginning. Next, the Boolean condition is evaluated. If true, execute loop body, execute update block, and evaluate again. If false, jump to the end.

This is equivalent to the following expression(which is the example above in while section)

```
Initialization  
while (BOOLEAN CONDITION) {  
    // Loop Body  
    Update  
}
```

Assembly for loop

eval:	INITIALIZATION BLOCK		eval:	LDS R18, i
	EXPRESSION EVALUATION			LDS R19, size
	CPI R18, 0			CP R18, R19
	BREQ done			BRGE done
				LDS R20, j
				CPI R20, 0
				BREQ done
(body:) Loop body			body:	Loop body
				
	Update			INC R18
	JMP eval	;;Evaluate again		JMP eval
done:			done:	

The evaluation of the Boolean condition may be done using the Short-circuit evaluation depending on the high level programming language. As it was the case for the while statement, once the initial code is produced, optimizing compilers will analyze the use of variables and registers and change instructions to improve the overall program performance.

ELEC 1601 Week 12

Subroutine Execution

Navigation

- CALL and RET Instructions
- Parameters and Result
- Activation Block

CALL and RET Instructions

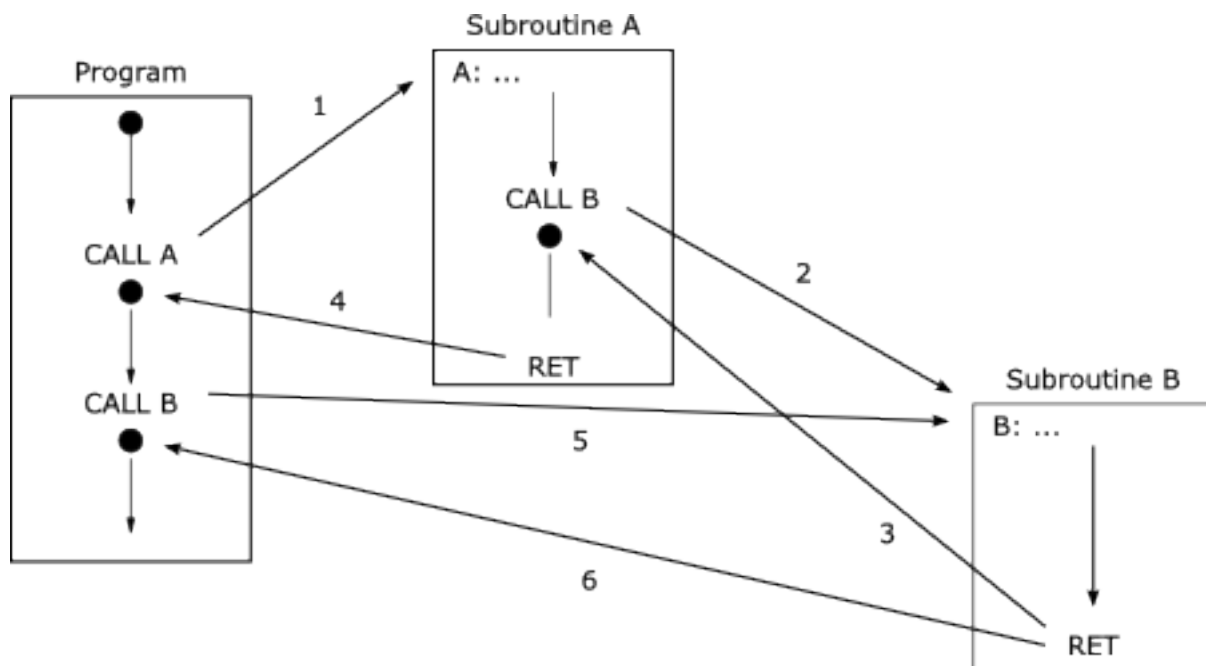
CALL is the simplest instruction to call a subroutine. The only operand of this instruction is a label corresponding with the first instruction of the subroutine.

After CALL instruction, the microprocessor **stores the PC in the stack** and **fetches the instruction in the memory position** corresponding with the **label** given as parameter.

The execution continues at that new point until the instruction RET (with no operands) is executed, at which point, the microprocessor pops the address from the stack and returns to the point in which the subroutine was called and fetches the instruction following the CALL (Jump to the next instruction after CALL).

The address used by the RET instruction is not a fixed value but it depends on the location of the previous CALL instruction.

Subroutine calls can be nested: a subroutine can be invoked in the middle of another subroutine to an arbitrary depth.



The instant in which the address to return from a subroutine call is known is precisely when it is being invoked. During the execution of the CALL instruction the program counter (before being modified with the address of the subroutine) points to the instruction following the CALL. That value is stored as the return address. But since there could be an arbitrary number of nested subroutine calls, we need a mechanism that allows the storage of the return addresses of those subroutines that are still in the middle of its invocation.

Stack trace: Sequence of functions when hit an exception.

Operating system calls the main.

Call main in main: stack grows bigger and bigger and goes back.

Stack overflow: stack run out of memory(part of memory in the microprocessor).

Mechanism to store the return address:

1. Register: A program with a deep chain of subroutine invocation would fill up the registers, the situation should be detected, and additional storage space would be required.
2. Memory: This mechanism is too restrictive for **recursive** subroutines (those that contain an self-invocation). For these routines, the instruction RET is executed several times with the same return address.
3. STACK: The crucial property satisfied by the return addresses is that they are required by the RET instruction in exactly the opposite order in which they are created. This property is derived from the nested invocation of subroutines. Whenever the instruction CALL is executed, a new return address is stored, and the next RET instruction requires the return address that was last created.

As a result of this technique, the CALL instruction changes the execution flow to the address given as parameter and pushes to the top of the stack the instruction next to the call in the program. Conversely, the instruction RET extracts the value at the top of the stack and stores it in the program counter, thus forcing the return of the program to the point right after the subroutine call.

The use of the stack is not restricted only to store the return address. Subroutines and programs may use the space to store variables, but with the restriction that when a subroutine executes its last instruction, the stack pointer must be pointing to the same memory address when the subroutine started.

The function with name **main** which contains the starting point for a program is also a subroutine. Programs start its execution with a return address stored on top of the stack, and that address is used upon termination to return the control to the operating system.

Parameters and Result

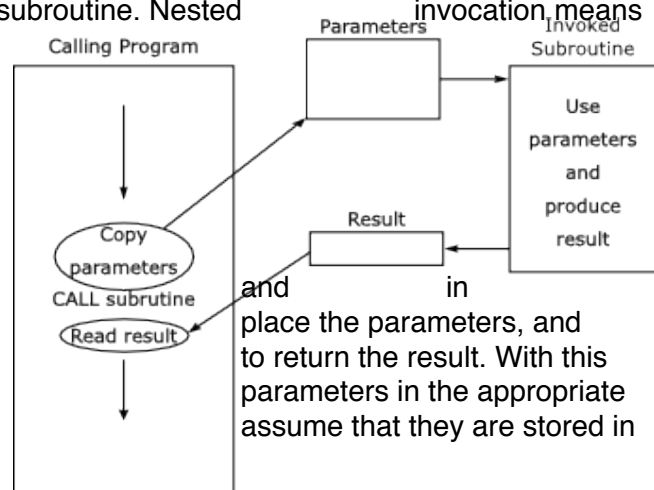
The parameters are copies of values that are made available to the subroutine by the caller, and are disposed of after the subroutine has finished. The returned result, on the other hand, is a value produced by subroutine and copied in a specific location so that the caller can use it.

A **calling convention** needs to be defined establishing the location of the spaces to store the parameters and the result.

From now on, the code portion containing the call to the subroutine will be referred to as the calling program, and the subroutine will be the invoked subroutine. Nested invocation means that an invoked subroutine can become temporarily the calling program for another subroutine.

1. Passing parameters and results through registers.

The calling convention defines which registers which order are used by the calling program to which register is used by the invoked subroutine convention, the calling program simply loads the registers, and the invoked subroutine may those registers at the start of its invocation.



The main disadvantage of this calling convention is that it provides a limited number of registers to store the parameters and results. High level programming languages do not impose any restriction in the number of arguments to a function, and therefore, the convention should include additional rules when there are not enough registers.

The main advantage of this convention is its efficiency. The program is already using the general purpose registers to store temporary values that are being manipulated. Leaving the parameters and collecting the result of a subroutine in some of them requires only simple operations (moving values from register to register).

2. Passing Parameters and Returning Result through Memory

Passing parameters and returning the result through memory consists on defining a memory area that is known to both the calling program and the invoked subroutine. This area will store the copy of the parameters and the space where the result will be placed.

The advantage of this technique with respect to the previous one is that the amount of space allocated to store the parameters can be bigger than when using registers.

In general, for a subroutine that receives n parameters and returns a result, a zone is defined with size equal to the total size of the parameters and the order in which the parameters are stored is also defined. This size is known at compilation time, and therefore, can be declared within the program.

The main disadvantage of this technique is that the memory space to store parameters and result needs to be previously defined, and there are additional instructions required to transfer the parameters from and to memory.

But this technique has two additional severe limitations. The first one is because most high level programming languages allow defining functions with an undetermined number of parameters. For this case, the memory area to store parameters cannot be defined as its size is unknown.

The second limitation is due to the presence of recursive functions. These functions invoke themselves, and therefore need to store an additional set of parameters and results for the new invocation while keeping the current copy of parameters. Recursive functions need as many spaces for the parameters as level of nesting of the function call. This nesting level is very likely to depend on the data given during execution, and therefore, there is no way to anticipate the number of these copies needed.

The key observation is that in the case of recursive functions, the parameters and result have exactly the same validity scope (which parameters are the right ones and when) than the return address of a subroutine. The return address itself can be considered an essential parameter given to the subroutine so that it can return to the appropriate code location. Thus, the most appropriate location to store parameters and result is the stack.

3. Passing Parameters and Returning Result through the Stack

Both parameters and results are temporary values that are only valid during a specific time in the program execution. This is precisely the purpose of the stack in a microprocessor. Furthermore, given a sequence of subroutine invocations, the order in which this space needs to be destroyed is exactly the opposite of the one in which they were created, as perfectly supported by the stack.

1. Result $\rightarrow$ Stack(first)
 2. Param $\rightarrow$ Stack (in reverse order)
- CALL
1. Remove param(POP R0)
 2. Retrieve result(POP Rx)

This technique then require the calling program to push to the stack all the arguments required by the subroutine plus some space to leave the result before executing the CALL instruction, and to remove that data from the stack afterwards. The memory portion created in the stack with this information is called the **activation block**.

The invoked subroutine needs to access the space in the stack created by the calling program. In principle, the content can be accessed using the stack pointer, because it contains the address of

the top element. But the stack pointer can suffer modifications as the code in the subroutine may push temporal values to the stack.

The technique to solve this problem is to store the value of the stack pointer in local registers right at the beginning of the subroutine and then use it to access parameters and space for the result using the data indirect with displacement addressing mode. This register is generically known as the **activation block pointer**.

PUSH R1 ; Result space	subroutine:	IN R31, 0x3E ; Stack Pointer
PUSH R19 ; Second param		IN R30, 0x3D ; Assign to Z
PUSH R18 ; First param		LDD R18, Z+3 ; Load first param
CALL subroutine		LDD R19, Z+4 ; Load second
POP R0 ; discard		...
POP R0 ; discard		STD Z+5, R20 ; Set Result
POP R20 ; Get result		OUT 0x3E, R31 ; Reload stack pointer
		OUT 0x3D, R30
		RET

The two initial instructions store the value of the stack pointer in register Z so that it can be used to access the parameter and result space.

We assume that the return address occupies 3 bytes in the stack. Thus, the first parameter is located 3 positions further the stack pointer ($Z + 3$ in the load instruction). Analogously, the second parameter is located 4 positions beyond the stack pointer ($Z + 4$) and the space for the result in the next position ($Z + 5$).

Activation Block

The data structure created in the stack for the purpose of subroutine execution is called the **activation block** and its construction and destruction is a shared responsibility between the calling program and the invoked subroutine. The calling program is in charge of allocating space for the result (if required), and load the parameters in a specific order. The CALL instruction automatically places the return address for the call at the top of the stack. In the AVR architecture the size of the return address stored in the stack by the CALL instruction is either 2 or 3 bytes depending on the processor model. Subroutine can access both parameters and result space with fixed displacement from the stack pointer.

Store local variables:

The space for these local variables is allocated by executing the instruction PUSH with any register and as many times as required by the size of the variables.

The activation block can also be seen as a complex data structure created specifically for the invocation of a subroutine with a fixed number of fields. Given the address at which this block is stored, we can use the data indirect with displacement addressing mode to access any of its fields, namely, local variables, parameters, and space for result.

The exact displacement to be used depends on the number and size of parameters, result and local variables needed, but all of them are known when translating a program, and therefore they can be fixed.

